

BÁO CÁO DỰ ÁN

Hệ thống SecureChat

NGUYỄN VŨ THÀNH

Thanh14704@gmail.com

Lĩnh Vực: Kỹ Sư Phần Mềm: Lập Trình Hệ Thống

Mentor hướng dẫn: Lê Thành Thân

Đơn vị: Viettel Cypher Security

Chương trình: Viettel Digital Talent

HÀ NỘI, 6/2026

TÓM TẮT NỘI DUNG DỰ ÁN

Dự án trình bày việc thiết kế và phát triển hệ thống SecureChat - một giải pháp truyền thông thời gian thực (Real-time Communication) theo mô hình Client-Server. Hệ thống được xây dựng hoàn toàn bằng ngôn ngữ C++ với mục tiêu đáp ứng đồng thời hai yêu cầu : Hiệu năng xử lý đồng thời cao ở mức cao (High Concurrency) và Bảo Mật thông tin đa lớp.

Ở phía Máy chủ (Server), kiến trúc hệ thống được thiết kế dựa trên mô hình điều khiển sự kiện bất đồng bộ (Event-Driven) sử dụng *epoll*, kết hợp cùng mô hình Thread Pool. Điều này cho phép Server xử lý bài toán C10K một cách tối ưu, không bị thất nút cổ chai tại các luồng I/O mạng. Cơ sở dữ liệu SQLite được tích hợp cùng cơ chế Write-Ahead Logging (WAL) giúp đảm bảo tốc độ truy vấn thông tin người dùng và lịch sử hệ thống trong môi trường đa luồng.

Về mặt Giao thức và Bảo mật, hệ thống không sử dụng các giao thức văn bản truyền thống mà lựa chọn sử dụng giao thức truyền tải nhị phân tùy chỉnh (Custom Binary Protocol) với Header chứa mã kiểm lỗi CRC32. Quá trình bảo mật áp dụng mô hình mã hóa lai (Hybrid Crypto): sử dụng RSA-2048 để trao đổi khóa phiên một cách an toàn ở bước bắt tay (Handshake), sau đó toàn bộ payload giao tiếp được mã hóa và xác thực tính nguyên vẹn bằng chuẩn AES-256-GCM.

Ngoài ra, dự án tập trung nghiên cứu và cài đặt một dịch vụ an ninh mạng chủ động ngay trong nhân máy chủ, bao gồm: module Hệ thống phát hiện xâm nhập (Intrusion Detector - IDS) để phân tích hành vi độc hại, bộ định tuyến giới hạn tốc độ (Rate Limiter) chống lại các cuộc tấn công DoS/Spam dựa trên thuật toán Token-Bucket, và Nhật ký kiểm tra (Audit Logger) lưu vết các thao tác nhạy cảm. Phía Client cài đặt giao diện dòng lệnh (CLI) nhẹ, hỗ trợ đa dạng tính năng nghiệp vụ từ nhắn tin cá nhân, phòng chat nhóm, truyền tải tập tin, đến các tập lệnh của quản trị viên (Admin Commands).

Kết quả đạt được là một nền tảng giao tiếp tin cậy, hoạt động ổn định và đáp ứng các tiêu chuẩn bảo mật thực tế, có khả năng mở rộng và tích hợp thêm các dịch vụ khác trong tương lai.

MỤC LỤC

Danh sách hình vẽ	iii
Danh sách bảng	iv
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Khảo sát hiện trạng	1
1.1.1 Đặt vấn đề	1
1.1.2 Mục tiêu và phạm vi của đề tài	2
1.2 Phân tích yêu cầu hệ thống	3
1.2.1 Yêu cầu chức năng	3
1.2.2 Yêu cầu phi chức năng	4
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	6
2.1 Kiến trúc tổng thể và Phân tích yêu cầu	6
2.1.1 Phân tích yêu cầu hệ thống	6
2.1.2 Kiến trúc tổng thể	8
2.2 Giao thức Mạng và Mô hình I/O	12
2.2.1 Mô hình Client-Server và I/O Multiplexing	12
2.2.2 Thiết kế Định dạng Gói tin	13
2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực	14
2.3.1 Cấu trúc Lưu trữ Dữ liệu	14
2.3.2 Luồng Xác thực và Quản lý Phiên	17
2.4 Giao thức Mật mã và Tính bảo mật	18
2.4.1 Luồng Trao đổi khóa	18
2.5 Hệ thống Chức năng Cốt lõi	20
2.5.1 Luồng dữ liệu Trò chuyện và Quản lý phòng	20
2.5.2 Trung chuyển Tập tin ẩn danh	22
2.6 Hệ thống Quản trị và Chống lạm dụng	24
2.6.1 Hàng rào Chống Spam và DDoS	24
2.6.2 Phân quyền RBAC và Quản trị	26
CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	28

3.1	Môi trường và Công nghệ Triển khai	28
3.1.1	Cấu hình Môi trường Hệ thống	28
3.1.2	Kịch bản biên dịch và vận hành	30
3.2	Kịch bản Kiểm thử và Đánh giá Hiệu năng	33
3.2.1	Tổng quan hệ thống Kiểm thử tự động	33
3.2.2	Đánh giá giới hạn chịu tải	35
3.3	Đánh giá Khả năng phòng thủ Hệ thống	37
3.3.1	Phòng thủ trước tấn công Từ chối dịch vụ	37
3.3.2	Phòng thủ trước tấn công Tiêm nhiễm và Vượt quyền	39
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		42
4.1	Kết quả đạt được	42
4.2	Hướng phát triển tương lai	43

Danh sách hình vẽ

Hình 2.1:	Sơ đồ Component Kiến trúc Tổng thể Hệ thống	8
Hình 2.2:	Sơ đồ Thực thể Liên kết của hệ thống CSDL	15
Hình 2.3:	Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên	18
Hình 2.4:	Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES	20
Hình 2.5:	Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng	22
Hình 2.6:	Máy trạng thái hữu hạn luồng Truyền tệp ẩn danh	24
Hình 2.7:	Lược đồ Khối của hệ thống Tường lửa mềm	26

Danh sách bảng

Bảng 2.1:	Đặc tả yêu cầu chức năng hệ thống	6
Bảng 2.2:	Đặc tả yêu cầu phi chức năng hệ thống	7
Bảng 2.3:	Phân rã các thành phần kiến trúc hệ thống	9
Bảng 2.4:	Cấu trúc Gói tin Nhị phân	14
Bảng 2.5:	Đặc tả các Bảng thực thể trong Cơ sở dữ liệu	16
Bảng 2.6:	Chiến lược cấp phát khóa trong RoomManager	21
Bảng 2.7:	Các trạng thái của Phiên truyền tệp	23
Bảng 2.8:	Hệ thống Điểm phạt Hành vi	25
Bảng 2.9:	Ma trận Phân quyền của Hệ thống	27
Bảng 3.1:	Bảng Ma trận Phụ thuộc Thư viện	29
Bảng 3.2:	Các tập lệnh và tham số cấu hình trong chu trình biên dịch	31
Bảng 3.3:	Các tham số dòng lệnh vận hành Máy chủ	32
Bảng 3.4:	Các tham số dòng lệnh kết nối Máy khách	33
Bảng 3.5:	Ma trận phân bổ các module kiểm thử tự động	35
Bảng 3.6:	Đo lường Thông lượng và Độ trễ của Server	37
Bảng 3.7:	Cấu hình giới hạn tốc độ của hệ thống	39
Bảng 3.8:	Ma trận điểm số rủi ro của hệ thống phòng thủ	41

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
ACK	Acknowledgment
AD	Authenticated Data
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
CLI	Command-Line Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DoS	Denial-of-Service
DDoS	Distributed Denial-of-Service
ERD	Entity-Relationship Diagram
ET	Edge-Triggered
FD	File Descriptor
GCM	Galois/Counter Mode
GUI	Graphical User Interface
HIDS	Host-based Intrusion Detection System
HMAC	Hash-based Message Authentication Code
I/O	Input/Output
IDS	Intrusion Detection System
IP	Internet Protocol
LT	Level-Triggered
MTU	Maximum Transmission Unit
OOM	Out of Memory
RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAL	Write-Ahead Logging

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Khảo sát hiện trạng

1.1.1 Đặt vấn đề

Sự mở rộng quy mô của hạ tầng mạng Internet và thiết bị di động đã gia tăng đáng kể nhu cầu giao tiếp trực tuyến. Trong bối cảnh đó, các ứng dụng truyền thông thời gian thực (Real-time Communication) đóng vai trò nền tảng trong việc duy trì luồng trao đổi thông tin liên tục và đa chiều. Thành phần cốt lõi của các hệ thống này là kiến trúc máy chủ (Chat Server) với năng lực xử lý cao, đảm nhiệm vai trò tiếp nhận, điều phối và định tuyến thông điệp song công (full-duplex) với độ trễ thấp giữa một số lượng lớn thiết bị máy khách (Client) phân tán.

Tuy nhiên, việc thiết kế và triển khai một kiến trúc mạng đáp ứng yêu cầu trên đối mặt với nhiều thách thức kỹ thuật phức tạp, trong đó trọng tâm là bài toán tối ưu hiệu năng. Khi hệ thống phải duy trì hàng chục nghìn kết nối đồng thời (bài toán C10K), mô hình cấp phát luồng theo kết nối (thread-per-client) truyền thống thường bộc lộ những hạn chế cố hữu, gây ra hiện tượng thắt cổ chai I/O mạng (Network I/O Bottleneck) và suy kiệt tài nguyên phần cứng. Thêm vào đó, việc sử dụng các giao thức dựa trên văn bản (Text-based Protocol) làm gia tăng chi phí phân tích cú pháp (parsing overhead) và hao phí băng thông. Những vấn đề này đặt ra yêu cầu cấp thiết trong việc xây dựng hệ thống dựa trên các mô hình I/O đa kênh tiên tiến (I/O Multiplexing), kết hợp cùng thiết kế cấu trúc gói tin nhị phân (Binary Protocol) nhằm giảm thiểu kích thước dữ liệu và tối ưu hóa triệt để độ trễ truyền tải.

Bên cạnh năng lực xử lý đồng thời, việc bảo mật luồng dữ liệu truyền tải là một yêu cầu kỹ thuật mang tính tiên quyết. Nhiều kiến trúc hệ thống hiện hành vẫn bộc lộ lỗ hổng trong cơ chế kiểm soát truy cập và bảo vệ kênh truyền, làm gia tăng nguy cơ đối mặt với các hình thức tấn công mạng như nghe lén (Eavesdropping), tấn công xen giữa (Man-in-the-Middle) hay cạn kiệt tài nguyên (DoS/Spam). Việc nghiên cứu và tích hợp các module phòng thủ chủ động - bao gồm mã hóa dữ liệu, hệ thống phát hiện xâm nhập (Intrusion Detection System - IDS) và cơ chế giới hạn lưu lượng (Rate Limiter) - là giải pháp thiết yếu để chuyển đổi một máy chủ truyền thông tiêu chuẩn thành một kiến trúc giao tiếp an toàn, khép kín và có độ tin cậy cao.

Xuất phát từ những thách thức kỹ thuật và nhu cầu thực tiễn nêu trên, việc nghiên cứu, thiết kế và phát triển một hệ thống “Secure Chat Server” hiệu năng cao là một định hướng cấp thiết. Đề tài đòi hỏi việc kiểm soát toàn diện kiến trúc hệ thống, từ phần tối

ưu hóa I/O mạng, định tuyến thông điệp Client-Server tốc độ cao, cho đến việc triển khai các cơ chế bảo mật chuyên sâu đa lớp. Hướng tiếp cận này không chỉ giải quyết trọn vẹn bài toán vận hành thực tế mà còn đóng góp giá trị học thuật cao trong lĩnh vực lập trình hệ thống và an toàn thông tin.

1.1.2 Mục tiêu và phạm vi của đề tài

1. Mục tiêu của đề tài

Đề tài hướng tới việc nghiên cứu, thiết kế và phát triển hoàn chỉnh một hệ thống máy chủ truyền thông (Secure Chat Server) hiệu năng cao, ứng dụng mô hình Client-Server với các mục tiêu cụ thể như sau:

- Tối ưu hóa hiệu năng xử lý đồng thời: Ứng dụng các công nghệ điều khiển sự kiện bất đồng bộ (I/O Multiplexing với epoll) và mô hình Thread Pool để giải quyết triệt để bài toán C10K, đảm bảo hệ thống duy trì hàng chục nghìn kết nối mạng với độ trễ tối thiểu và không xảy ra hiện tượng thất cổ chai I/O.
- Thiết kế giao thức truyền tải tối ưu: Xây dựng một giao thức gói tin nhị phân tùy chỉnh (Custom Binary Protocol) nhằm loại bỏ chi phí phân tích cú pháp (parsing overhead) và tiết kiệm băng thông mạng so với các giao thức dựa trên văn bản thông thường.
- Tích hợp hệ sinh thái an ninh mạng đa lớp: Triển khai mô hình mã hóa lai (Hybrid Crypto) sử dụng RSA-2048 để trao đổi khóa và AES-256-GCM để mã hóa tải trọng (payload). Đồng thời, xây dựng các module bảo mật chủ động trực tiếp trong nhân máy chủ bao gồm: Hệ thống phát hiện xâm nhập (IDS), Cơ chế giới hạn lưu lượng (Rate Limiter sử dụng thuật toán Token-Bucket) và Hệ thống nhật ký kiểm toán (Audit Logger).
- Đảm bảo tính nhất quán dữ liệu: Sử dụng hệ quản trị cơ sở dữ liệu SQLite kết hợp với cơ chế Write-Ahead Logging (WAL) để đảm bảo an toàn dữ liệu và tối ưu tốc độ truy vấn trong môi trường đa luồng.

2. Phạm vi nghiên cứu và triển khai

Trong khuôn khổ của đồ án, phạm vi nghiên cứu và triển khai được giới hạn ở các khía cạnh sau:

- Phạm vi kiến trúc hệ thống: Tập trung chuyên sâu vào việc xây dựng kiến trúc lõi của phần mềm Máy chủ (Server-side) bằng ngôn ngữ C++ hiện đại. Phần mềm Máy khách (Client-side) được phát triển dưới dạng giao diện dòng lệnh

(CLI), đóng vai trò như một công cụ tương tác thực tế để kiểm chứng hiệu quả của các tính năng nghiệp vụ.

- Phạm vi tính năng nghiệp vụ Hệ thống: triển khai đầy đủ các tính năng giao tiếp cốt lõi bao gồm: Nhắn tin cá nhân (Direct Message), Nhắn tin và quản lý phòng trò chuyện nhóm (Rooms), Truyền tải tập tin (File Transfer) và các Tập lệnh quản trị viên (Admin Commands).
- Phạm vi đánh giá và kiểm định: Đề tài tập trung đo lường năng lực xử lý (benchmark) dưới điều kiện tải cao và mô phỏng các kịch bản tấn công mạng cơ bản (như DoS, quét cổng, giả mạo gói tin) nhằm kiểm chứng năng lực phòng thủ của các module an ninh. Đề tài không mở rộng nghiên cứu sang việc phát triển giao diện đồ họa người dùng (GUI) hay các nền tảng ứng dụng di động.

1.2 Phân tích yêu cầu hệ thống

1.2.1 Yêu cầu chức năng

Dựa trên mục tiêu xây dựng một nền tảng trò chuyện bảo mật và hiệu năng cao, hệ thống cần đáp ứng các nhóm yêu cầu chức năng cốt lõi sau đây:

1. Quản lý Tài khoản và Xác thực (Authentication & Authorization):

- Cung cấp tính năng Đăng ký (Register) và Đăng nhập (Login) an toàn. Mật khẩu người dùng không lưu bản rõ mà phải qua các hàm băm mật mã (như SHA256/bcrypt) để đảm bảo an toàn cơ sở dữ liệu.
- Cấp phát thẻ định danh phiên (Session Token) sử dụng chữ ký điện tử HMAC, hỗ trợ cơ chế tự động phục hồi kết nối (Session Resumption) giúp người dùng giữ nguyên phiên đăng nhập khi mạng chập chờn.

2. Quản lý phiên kết nối và Trạng thái (Session Management):

- Quản lý vòng đời socket (Connect, Disconnect) và duy trì tín hiệu sống sót thông qua cơ chế nhện tim (Ping/Pong).
- Hỗ trợ tra cứu danh sách người dùng đang trực tuyến (List Users) và xem thông tin hồ sơ của một tài khoản cụ thể (Whois).

3. Giao tiếp và Nhắn tin thời gian thực (Realtime Chatting):

- Nhắn tin cá nhân (Direct Message): Cho phép trao đổi thông điệp trực tiếp 1-1 với độ trễ tối thiểu.

- Nhắn tin nhóm (Room Chat): Hỗ trợ danh sách phòng trò chuyện (List Rooms), cho phép người dùng tạo phòng mới (Create Room), tham gia (Join), rời phòng (Leave) và gửi tin nhắn quảng bá (Broadcast) tới toàn bộ thành viên.
- Hỗ trợ truy vấn lịch sử trò chuyện đã được lưu trữ bền vững trong cơ sở dữ liệu.

4. Truyền tải tập tin (File Transfer):

- Cung cấp quy trình gửi/nhận tập tin hai chiều với luồng kiểm soát rõ ràng: gửi yêu cầu truyền (File Request) và chờ phía nhận xác nhận (Accept/Reject).
- Dữ liệu tập tin được chia nhỏ và truyền theo từng khối (Chunking) để tránh việc một tập tin dung lượng lớn chiếm dụng và gây tắc nghẽn luồng xử lý thông điệp chính của hệ thống.

5. Quản trị hệ thống (Admin Commands):

- Phân tách vai trò và cấp quyền Quản trị viên (Admin Privilege) để điều hành cộng đồng.
- Tích hợp bộ công cụ xử lý vi phạm: kích xuất người dùng (Kick), cấm gửi tin nhắn (Mute), cấm truy cập vĩnh viễn (Ban IP/Username), thăng cấp quyền (Promote) và gửi thông báo khẩn cấp tới toàn server (System Broadcast).

6. Cơ chế phòng vệ và An ninh chủ động (Security):

- Bắt tay mật mã (Crypto Handshake): Tự động trao đổi khóa (Key Exchange) ngay khi khởi tạo kết nối để thiết lập kênh truyền mã hóa bảo mật trước khi cho phép trao đổi dữ liệu nghiệp vụ.
- Ngăn chặn lạm dụng (Rate Limiting): Áp dụng thuật toán giới hạn tốc độ nhằm chống lại các nỗ lực tấn công từ chối dịch vụ (DoS) hoặc nhồi nhét tin nhắn rác (Spam).
- Hệ thống phát hiện xâm nhập (IDS): Tự động ghi nhận và phân loại các vi phạm (sai mật khẩu, quét cổng, gửi gói tin dị dạng) để trừ điểm rủi ro, từ đó ra quyết định cấm (Ban) tạm thời hoặc vĩnh viễn đối với các IP độc hại.

1.2.2 Yêu cầu phi chức năng

Bên cạnh các tính năng nghiệp vụ cốt lõi, hệ thống đặc biệt chú trọng vào các yêu cầu phi chức năng nhằm đảm bảo năng lực vận hành, độ tin cậy và an toàn thông tin toàn diện trong môi trường thực tế:

1. Hiệu năng và Khả năng mở rộng (Performance & Scalability):

- Khả năng xử lý đồng thời: Kiến trúc máy chủ phải được thiết kế dựa trên cơ chế I/O đa kênh (I/O Multiplexing) thông qua API epoll kết hợp với mô hình Thread Pool. Thiết kế này đảm bảo luồng điều phối chính không bao giờ bị nghẽn (non-blocking), giải quyết hiệu quả bài toán C10K với khả năng duy trì hàng chục nghìn kết nối mà vẫn giữ độ trễ ở mức tối thiểu.
- Tối ưu hóa băng thông truyền tải: Áp dụng giao thức mạng nhị phân tùy chỉnh (Custom Binary Protocol) thay thế cho văn bản thuần túy nhằm tối thiểu hóa kích thước gói tin, tiết kiệm băng thông và giảm thiểu hao phí CPU trong quá trình phân tích cú pháp (parsing).

2. Bảo mật và Toàn vẹn dữ liệu (Security & Integrity):

- Mã hóa kênh truyền: Toàn bộ lưu lượng mạng lưới phải được mã hóa tải trọng theo tiêu chuẩn công nghiệp (AES-256-GCM), với khóa phiên (Session Key) được thỏa thuận và phân phối an toàn thông qua cơ chế mã hóa khóa công khai (RSA-2048).
- Xác thực tính nguyên vẹn: Sử dụng mã kiểm lỗi (CRC32) và mã xác thực thông điệp (HMAC) trên từng gói tin để ngăn chặn mọi hành vi đánh tráo, sửa đổi trái phép dữ liệu trên đường truyền.
- An toàn lưu trữ dữ liệu: Mật khẩu người dùng bắt buộc phải được băm một chiều an toàn trước khi lưu. Hệ thống cơ sở dữ liệu nội bộ (SQLite) phải áp dụng cơ chế ghi chép trước nhật ký (Write-Ahead Logging) để duy trì tính nhất quán, không làm hỏng file dữ liệu ngay cả khi hệ thống gặp sự cố đột ngột.

3. Tính tin cậy và Khả năng chịu lỗi (Reliability & Fault Tolerance):

- Quản lý tài nguyên chặt chẽ: Tự động dọn dẹp và thu hồi toàn bộ tài nguyên (socket, phiên đăng nhập, vùng nhớ cấp phát) của các kết nối chết, ngắt kết nối đột ngột hoặc vi phạm thời gian chờ (Timeout). Quy trình này giúp triệt tiêu hoàn toàn rủi ro cạn kiệt bộ mô tả tệp (File Descriptor Exhaustion) và rò rỉ bộ nhớ (Memory Leak).
- Khả năng phục hồi luồng dữ liệu mạng: Xây dựng bộ đệm mạng (Network Buffer) độc lập cho từng kết nối với năng lực tự động xử lý các rủi ro đặc thù của luồng truyền TCP, bao gồm hiện tượng phân mảnh gói tin (TCP Fragmentation) và dính gói (TCP Sticky Packets), đảm bảo quá trình đọc/ghi dữ liệu luôn trọn vẹn và không gây sập đổ tiến trình.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày chi tiết kiến trúc tổng thể và các quyết định thiết kế cốt lõi của hệ thống. Quá trình phân tích và thiết kế được gắn kết chặt chẽ với các nền tảng lý thuyết về mạng máy tính, mô hình vào/ra (I/O) và mật mã học, nhằm làm rõ cơ chế vận hành của từng phân hệ (module) trong kiến trúc thực tế.

2.1 Kiến trúc tổng thể và Phân tích yêu cầu

2.1.1 Phân tích yêu cầu hệ thống

Trong quá trình phát triển hệ thống truyền thông thời gian thực (Real-time Communication), phân tích yêu cầu là giai đoạn tiên quyết nhằm xác định năng lực xử lý và kiến trúc của máy chủ. Khác với mô hình tương tác yêu cầu - phản hồi (Request - Response) truyền thống, máy chủ trò chuyện (Chat Server) không chỉ cần đáp ứng các yêu cầu chức năng (Functional Requirements) mà còn phải tuân thủ nghiêm ngặt các yêu cầu phi chức năng (Non-Functional Requirements) như khả năng chịu tải cao, độ trễ thấp và bảo mật thông tin.

1. Yêu cầu chức năng

Hệ thống được thiết kế để phục vụ hai nhóm tác nhân (Actors) chính: Người dùng (User) và Quản trị viên (Admin). Đặc tả chi tiết các nhóm tính năng cốt lõi được trình bày tại Bảng 2.1.

Bảng 2.1: Đặc tả yêu cầu chức năng hệ thống

Nhóm chức năng	Tác nhân	Mô tả chi tiết
Xác thực định danh	Người dùng	Cung cấp cơ chế đăng ký và đăng nhập qua kênh truyền bảo mật. Quản lý vòng đời phiên làm việc (Session Token) nhằm duy trì phiên xác thực của người dùng.
Nhắn tin trực tiếp	Người dùng	Hỗ trợ trao đổi thông điệp song công (Full-duplex) giữa hai người dùng (Direct Message).

Nhóm chức năng	Tác nhân	Mô tả chi tiết
Quản lý phòng trò chuyện	Người dùng	Khởi tạo, tham gia và quản lý trạng thái phòng trò chuyện (Rooms). Tích hợp cơ chế kiểm soát truy cập bằng mật khẩu cho các phòng riêng tư (Private Rooms).
Truyền tải tệp tin	Người dùng	Hỗ trợ truyền nhận tệp đa phương tiện thông qua cơ chế phân mảnh dữ liệu (Data Chunking), đảm bảo tính toàn vẹn dữ liệu trong quá trình truyền tải.
Quản trị hệ thống	Quản trị viên	Vận hành cơ chế kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) để quản lý tài khoản người dùng và giám sát trạng thái hệ thống.
Xử lý vi phạm	Quản trị viên	Thực thi các biện pháp kiểm soát: cấm gửi tin (Mute), ngắt kết nối (Kick), hoặc khóa tài khoản (Ban) đối với các hành vi vi phạm chính sách hoặc phát tán mã độc.

2. Yêu cầu phi chức năng

Yêu cầu phi chức năng xác định các ràng buộc kỹ thuật đối với kiến trúc lõi, nhằm đảm bảo tính sẵn sàng, độ tin cậy và an toàn của hệ thống mạng.

Bảng 2.2: Đặc tả yêu cầu phi chức năng hệ thống

Tiêu chí	Mô tả chi tiết
Hiệu năng và bài toán C10K	Duy trì tính ổn định dưới tải trọng cao với số lượng lớn kết nối đồng thời. Yêu cầu ứng dụng mô hình I/O không chặn (Non-blocking I/O) và tối ưu hóa luồng xử lý.
Bảo mật và Mã hóa	Mã hóa toàn bộ lưu lượng mạng ở chế độ đầu cuối (End-to-End). Đảm bảo tính bí mật (Confidentiality) và tính toàn vẹn (Integrity) của luồng dữ liệu dựa trên các chuẩn mật mã hiện đại.

Tiêu chí	Mô tả chi tiết
Phòng chống tấn công mạng	Tích hợp cơ chế giám sát và giới hạn lưu lượng (Rate Limiting) nhằm giảm thiểu rủi ro từ các cuộc tấn công từ chối dịch vụ (DoS), thư rác (Spam) và chèn mã độc.
Nhật ký và Kiểm toán	Ghi nhận toàn vẹn các sự kiện bảo mật, phân loại mức độ rủi ro và cung cấp dữ liệu định dạng chuẩn phục vụ quá trình kiểm toán (Audit Logging).

2.1.2 Kiến trúc tổng thể (System Architecture)

Cơ sở lý thuyết để xây dựng kiến trúc hệ thống mạng máy chủ là mô hình Kiến trúc phân tầng (Layered Architecture) kết hợp với nguyên lý Tách biệt trách nhiệm (Separation of Concerns). Trong kỹ nghệ phần mềm, việc phân tách hệ thống thành các tầng độc lập giúp cô lập các rủi ro phát sinh, giảm thiểu sự phụ thuộc lẫn nhau (Coupling) và gia tăng tính gắn kết nội bộ (Cohesion) của từng phân hệ. Đối với các ứng dụng truyền thông thời gian thực, mô hình này cho phép tách biệt luồng xử lý I/O mạng cấp thấp khỏi các thuật toán nghiệp vụ cấp cao, từ đó tối ưu hóa khả năng mở rộng (Scalability) và đơn giản hóa quá trình bảo trì mã nguồn.

Dựa trên nền tảng lý thuyết đó, kiến trúc tổng thể của hệ thống được thiết kế theo mô hình hướng thành phần (Component-Based) và được cấu trúc thành ba phân tầng chính. Quá trình xử lý vòng đời của một thông điệp (Message Lifecycle) sẽ đi tuần tự từ khâu tiếp nhận dạng luồng byte thô cho đến khâu thực thi nghiệp vụ cuối cùng.

Hình 2.1: Sơ đồ Component Kiến trúc Tổng thể Hệ thống

Việc ánh xạ từ mô hình lý thuyết sang kiến trúc thực tế được thể hiện qua các khối mã nguồn cốt lõi (Codebase). Bảng 2.3 trình bày tổng quan sự phân rã của các module tương ứng với từng phân tầng.

Bảng 2.3: Phân rã các thành phần kiến trúc hệ thống

Phân tầng	Thành phần	Chức năng cốt lõi
Tầng Mạng và Cốt lõi	TcpServer	Cổng giao tiếp, quản lý vòng đời kết nối TCP.
	EventLoop	Điều phối sự kiện mạng phi đồng bộ (I/O Multiplexing).
	ThreadPool	Quản lý nhóm luồng thực thi tác vụ tính toán chuyên sâu.
	ClientSession	Lưu trữ và duy trì trạng thái của từng phiên kết nối khách.
Tầng Bảo mật và Lọc	KeyExchange	Đàm phán khóa mã hóa lai (Handshake).
	CryptoEngine	Thực thi mã hóa và giải mã luồng dữ liệu truyền tải.
	RateLimiter	Giới hạn tần suất yêu cầu, ngăn chặn hành vi ngập lụt mạng.
	IntrusionDetector	Dò tìm các mẫu dấu hiệu tấn công đã biết (Signatures).
	MessageFilter	Sàng lọc tải trọng (Payload), loại bỏ các ký tự độc hại.
	AuditLogger	Ghi nhận sự kiện bảo mật, phục vụ truy xuất kiểm toán.
Tầng Nghiệp vụ	AuthManager	Kiểm soát định danh, xác thực và cấp phát mã phiên (Session).
	RoomManager	Quản lý không gian trò chuyện chung.
	Room	Xử lý đa nhiệm, phân phối thông điệp tới thành viên.

Phân tầng	Thành phần	Chức năng cốt lõi
	FileTransfer	Điều phối luồng dữ liệu trung chuyển tệp tin đa phương tiện.
	AdminCommands	Xử lý tập lệnh đặc quyền quản trị hệ thống.

Chi tiết về thiết kế thực tiễn và cơ chế phối hợp giữa các thành phần trong từng phân tầng được trình bày cụ thể như sau:

1. Tầng Mạng và Cốt lõi (Network & Core Layer)

Đây là lớp thấp nhất chịu trách nhiệm tương tác trực tiếp với giao diện giao tiếp mạng (Socket API) của hệ điều hành. Lớp này ứng dụng mô hình sự kiện bất đồng bộ nhằm loại bỏ hoàn toàn các rào cản về thất cổ chai I/O.

- **TcpServer:** Đóng vai trò là cổng vào duy nhất (Entry Point) của máy chủ. Module này liên tục lắng nghe tại một cổng mạng được chỉ định, tiếp nhận các yêu cầu kết nối TCP từ máy khách, sau đó tiến hành khởi tạo và cấp phát các cấu trúc dữ liệu phiên làm việc tương ứng.
- **EventLoop:** Đóng vai trò là hạt nhân của tầng mạng, triển khai mô hình I/O Multiplexing tiên tiến. Nó đảm nhiệm việc giám sát đồng thời hàng loạt các mô tả tệp (File Descriptor), thu nhận sự kiện mạng (đọc/ghi dữ liệu) và phân phối luồng xử lý đến các hàm gọi lại (Callbacks) một cách phi đồng bộ.
- **ThreadPool:** Cung cấp cơ chế tái sử dụng tài nguyên tính toán thông qua một nhóm các luồng được khởi tạo sẵn (Pre-instantiated Threads). Các tác vụ chuyên sâu về CPU, chẳng hạn như tính toán khóa mật mã, được đẩy vào hàng đợi của ThreadPool để thực thi song song, đảm bảo không gây ách tắc cho vòng lặp sự kiện chính.
- **ClientSession:** Là đại diện logic cho một kết nối vật lý đã được thiết lập. Module này quản lý vòng đời độc lập của từng máy khách, lưu trữ định danh mạng, bộ đệm dữ liệu riêng biệt và duy trì liên kết với khóa mã hóa phiên của hệ thống bảo mật.

2. Tầng Bảo mật và Lọc (Security & Filter Layer)

Được thiết kế dựa trên nguyên lý phòng thủ chiều sâu (Defense-in-Depth), lớp bảo mật đóng vai trò như một bức tường lửa nội tại. Toàn bộ dòng dữ liệu luân chuyển lên từ tầng mạng đều phải đi qua hàng rào kiểm duyệt tự động tại lớp này.

- **KeyExchange và CryptoEngine:** Bộ đôi module chuyên biệt về an toàn mật mã. KeyExchange xử lý cơ chế thỏa thuận khóa công khai/bí mật trong giai đoạn khởi tạo (Handshake), trong khi CryptoEngine thực thi quá trình giải mã hai chiều đối với nội dung thông điệp bằng các thuật toán khối đối xứng tốc độ cao.
- **RateLimiter:** Áp dụng các thuật toán điều tiết lưu lượng để giám sát và hạn chế tần suất truy vấn từ mỗi kết nối. Module này chủ động cắt giảm các luồng dữ liệu dư thừa, bảo vệ hệ thống trước nguy cơ cạn kiệt tài nguyên do thư rác (Spam/DoS).
- **IntrusionDetector và MessageFilter:** Thành phần phân tích bảo mật mức ứng dụng. Sau khi dữ liệu được giải mã, nó sẽ được quét qua IntrusionDetector để dò tìm các mẫu dấu hiệu tấn công đã biết. Các ký tự bất hợp lệ sẽ tiếp tục bị MessageFilter loại bỏ nhằm đảm bảo tính toàn vẹn của dữ liệu trước khi xử lý logic.
- **AuditLogger:** Quản lý tiến trình lưu vết thông tin. Bất kỳ cảnh báo bảo mật nào phát sinh từ các bộ lọc đều được module này ghi nhận lại một cách toàn vẹn, đi kèm theo dấu thời gian (Timestamp) và địa chỉ IP gốc phục vụ truy xuất sau sự cố.

3. Tầng Nghiệp vụ (Business Logic Layer)

Lớp cao nhất quản lý toàn bộ các tính năng dịch vụ cốt lõi nhằm đáp ứng yêu cầu giao tiếp của người dùng. Do dữ liệu ở tầng này đã được chuẩn hóa và loại bỏ các yếu tố rủi ro, các module nghiệp vụ hoàn toàn tập trung vào việc xử lý logic thuần túy.

- **AuthManager:** Phụ trách đối chiếu thông tin định danh của người dùng với cơ sở dữ liệu. Nó thực thi luồng công việc đăng ký, kiểm chứng độ phức tạp của mật khẩu và quản lý cấp phát các thẻ phiên đăng nhập (Session Token) hợp lệ.
- **RoomManager và Room:** Cung cấp nền tảng quản lý không gian trò chuyện chung. RoomManager duy trì danh sách tham chiếu đa cấu trúc đến mọi phòng hiện có, trong khi Room đóng vai trò là một không gian đa nhiệm độc lập, xử lý các thao tác tham gia, rời bỏ và tự động phân phối thông điệp (Multicast) tới toàn bộ thành viên trực tuyến.
- **FileTransfer:** Phân hệ trung chuyển lưu lượng chuyên biệt, chịu trách nhiệm tái tổ hợp các mảnh dữ liệu nhị phân thô thành các tập tin hoàn chỉnh. Nó cung cấp cơ chế tải lên và tải xuống an toàn, được cô lập khỏi luồng xử lý thông điệp văn bản tiêu chuẩn.

- **AdminCommands:** Kênh thực thi các chính sách quản trị cấp cao. Module này sở hữu đặc quyền vô hiệu hóa phiên kết nối ngang hàng (Kick, Ban) hoặc điều chỉnh cấu hình định tuyến của bất kỳ người dùng nào đang tồn tại trong hệ thống, nhằm duy trì trật tự mạng.

2.2 Giao thức Mạng và Mô hình I/O

2.2.1 Mô hình Client-Server và I/O Multiplexing (epoll)

1. Cơ sở lý thuyết về I/O Multiplexing và Bài toán C10K

Kiến trúc Client-Server (C/S) là nền tảng cốt lõi của các ứng dụng truyền thông mạng. Trong môi trường hoạt động thực tế, khi một hệ thống phân tán phải đối mặt với bài toán C10K (xử lý đồng thời 10,000 kết nối), mô hình lập trình mạng truyền thống như cấp phát luồng cho từng kết nối (Thread-per-connection) sẽ nhanh chóng gây suy kiệt tài nguyên hệ điều hành. Chi phí chuyển đổi ngữ cảnh (Context Switching) giữa hàng vạn luồng sẽ tạo ra nút thắt cổ chai (Bottleneck) nghiêm trọng tại nhân (Kernel) hệ điều hành.

Để giải quyết triệt để bài toán này, lý thuyết mạng đề xuất kỹ thuật Đa hợp vào/ra (I/O Multiplexing). Kỹ thuật này cho phép một luồng (Thread) duy nhất có khả năng giám sát trạng thái của hàng loạt mô tả tệp (File Descriptor - FD). Trong hệ sinh thái Linux, `epoll` là giao diện lập trình ứng dụng (API) tiên tiến nhất, thay thế cho các cơ chế `select` hoặc `poll` cổ điển. Khác với phương pháp duyệt tuyến tính $O(N)$ tốn kém, `epoll` ứng dụng cấu trúc dữ liệu cây Đỏ-Đen (Red-Black Tree) để quản lý FD và danh sách liên kết cho các sự kiện sẵn sàng. Cơ chế hướng sự kiện (Event-driven) này giúp giảm độ phức tạp thuật toán xuống $O(1)$, mang lại hiệu năng ổn định bất chấp số lượng kết nối đang mở.

2. Triển khai thực tiễn trong mã nguồn hệ thống

Áp dụng nền tảng lý thuyết trên, hệ thống máy chủ (TcpServer) được thiết kế hoàn toàn theo mô hình I/O không chặn (Non-blocking I/O) kết hợp với `epoll` để điều phối các luồng kết nối TCP. Quá trình này được triển khai tuần tự qua các khối lệnh cốt lõi:

- **Khởi tạo và lắng nghe:** Hệ thống sử dụng các lệnh Socket cơ sở (`socket`, `bind`, `listen`) để tạo ra một Socket thụ động (`m_listenFd`) và ép buộc thuộc tính Non-blocking thông qua hàm `fcntl`. Socket này có nhiệm vụ chuyên biệt là tiếp nhận các yêu cầu kết nối (SYN) từ máy khách.

- **Quản trị bộ điều khiển sự kiện:** Lớp TcpServer khởi tạo một phiên bản epoll thông qua lệnh `epoll_create1(0)`. Socket lắng nghe ngay sau đó được đăng ký vào bộ điều khiển `m_epollFd` thông qua `epoll_ctl` để giám sát sự kiện có luồng kết nối mới (EPOLLIN).
- **Vòng lặp sự kiện (Event Loop):** Tại hàm `epollLoop()`, hệ thống duy trì một vòng lặp liên tục gọi `epoll_wait` để thu nhận tín hiệu từ Kernel. Tùy thuộc vào bản chất sự kiện, luồng xử lý sẽ được định tuyến như sau:
 - *Sự kiện trên Socket lắng nghe:* Máy chủ thực thi lệnh `accept` để thiết lập kết nối, tạo mới đối tượng `ClientSession`, cấu hình Non-blocking và đưa Socket mới (`clientFd`) vào danh sách giám sát của epoll kèm cờ EPOLLIN | EPOLLRDHUP. Đồng thời, tầng bảo mật (`RateLimiter`, `IntrusionDetector`) được kích hoạt để kiểm chứng địa chỉ IP.
 - *Sự kiện trên Socket dữ liệu:* Tín hiệu EPOLLIN cho biết có luồng byte đến từ máy khách. TcpServer tiến hành tiếp nhận dữ liệu thô thông qua `recv` và chuyển tiếp toàn bộ vùng đệm sang cho `ClientSession` để lắp ráp gói tin.

2.2.2 Thiết kế Định dạng Gói tin (Packet Format)

1. Cơ sở lý thuyết về thiết kế giao thức mạng

Trong hệ thống phân tán, luồng dữ liệu truyền tải qua giao thức TCP được thiết kế dưới dạng luồng byte liên tục (Byte Stream) không có ranh giới rõ ràng. Hiện tượng này tất yếu dẫn đến hai vấn đề kinh điển: chia cắt gói tin (Half-packet) và dính liền gói tin (Sticky-packet). Để máy thu có thể bóc tách thông điệp một cách chuẩn xác, giao thức ở tầng Ứng dụng (Application Layer) phải định nghĩa một cấu trúc ranh giới tĩnh. Hơn nữa, so với các giao thức dạng văn bản (Text-based protocol) như HTTP, việc tự định nghĩa một giao thức nhị phân (Binary Protocol) giúp tối ưu hóa chi phí phân tích cú pháp (Parsing overhead) và giảm thiểu băng thông.

2. Cấu trúc gói tin thực tế của hệ thống

Dựa trên lý thuyết đóng gói, hệ thống sử dụng một Giao thức Nhị phân (Custom Binary Protocol) có tính tinh gọn cao. Một khối thông điệp truyền tải (Packet) được phân chia thành hai phân đoạn chính, minh họa trong Bảng 2.4.

Bảng 2.4: Cấu trúc Gói tin Nhị phân (Binary Packet Format)

Phân đoạn	Kích thước	Trường dữ liệu	Mô tả chi tiết
Header	4 Bytes	msg_type (Loại tin nhắn)	Định nghĩa nghiệp vụ của gói tin (ví dụ: đăng nhập, trò chuyện, truyền tệp).
	4 Bytes	length (Kích thước Payload)	Lưu trữ độ dài chính xác của phần Payload, giải quyết triệt để lỗi Sticky-packet.
Payload	Tùy biến (Động)	Dữ liệu JSON đã mã hóa	Đóng gói tham số nghiệp vụ linh hoạt và được mã hóa nguyên khối (AES-256).

Quá trình luân chuyển và bóc tách gói tin giữa Client và Server diễn ra theo chu trình tuần tự như sau:

- **Đóng gói và mã hóa (Tại Client):** Khởi tạo đối tượng JSON, tuần tự hóa (Serialization) thành chuỗi, tiến hành mã hóa luồng dữ liệu, tính toán độ dài và đính kèm 8 Bytes Header nhị phân vào trước Payload.
- **Tiếp nhận khối Header (Tại Server):** ClientSession đọc chính xác 8 Bytes đầu tiên từ luồng TCP để bóc tách tham số msg_type và length.
- **Cấp phát và nạp Payload:** Dựa trên kích thước length, hệ thống phân bổ vùng nhớ đệm vừa đủ và nạp liên tục cho đến khi thu thập trọn vẹn phần Payload.
- **Giải mã và định tuyến:** CryptoEngine giải mã để khôi phục cấu trúc JSON nguyên thủy, sau đó TcpServer dựa vào msg_type để định tuyến tới các hàm xử lý chuyên biệt (như handleChatPrivate, handleRoomJoin).

2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực

2.3.1 Cấu trúc Lưu trữ Dữ liệu

1. Cơ sở lý thuyết về Tổ chức Cơ sở dữ liệu

Trong thiết kế hệ thống phần mềm, Cơ sở dữ liệu (Database) đóng vai trò là xương sống đảm bảo tính bền vững (Durability) của toàn bộ trạng thái hệ thống. Các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) thường được thiết lập để tuân thủ nghiêm ngặt các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) nhằm duy trì tính toàn vẹn thông tin. Quá trình thiết kế Lược đồ dữ liệu (Database Schema) đòi hỏi kỹ sư phải ứng dụng các quy tắc Chuẩn hóa (Normalization), đạt tối thiểu Dạng chuẩn 3 (3NF) nhằm loại bỏ triệt để hiện tượng dị thường khi cập nhật (Update Anomalies) và giảm thiểu không gian dư thừa lưu trữ.

Bên cạnh đó, trong bối cảnh máy chủ xử lý truyền thông thời gian thực với lưu lượng truy xuất I/O lớn từ hàng ngàn luồng kết nối đồng thời, cấu trúc lưu trữ cần một cơ chế khóa (Locking) tinh vi. Kỹ thuật Ghi trước Nhật ký (Write-Ahead Logging - WAL) là một giải pháp lý thuyết kinh điển giúp tách biệt luồng Đọc và Ghi, cho phép nhiều giao dịch truy vấn đọc được thực thi đồng thời mà không bị treo (Block) bởi một giao dịch ghi đang thao tác.

2. Triển khai lược đồ cơ sở dữ liệu trong hệ thống

Đáp ứng các tiêu chuẩn lý thuyết khắt khe, hệ thống đã tích hợp thư viện SQLite3 dưới vai trò là một Engine RDBMS nội tại, loại bỏ hoàn toàn chi phí truyền tải qua mạng nội bộ (IPC overhead) đến một máy chủ CSDL bên ngoài. Ngay tại khâu khởi tạo kết nối (`Database::open()`), hệ thống đã kích hoạt chỉ thị `PRAGMA journal_mode=WAL` để tối ưu hóa hiệu năng Đọc/Ghi đa luồng, đồng thời bật `PRAGMA foreign_keys=ON` nhằm giao phó hoàn toàn trách nhiệm duy trì tính Toàn vẹn tham chiếu (Referential Integrity) cho nhân xử lý của SQLite.

Toàn bộ cấu trúc lưu trữ được phân rã thành năm bảng thực thể chính. Bảng 2.5 minh họa chi tiết chức năng, cấu trúc khóa và các ràng buộc dữ liệu (Constraints) của từng bảng tương ứng với mã nguồn thực tế.

Hình 2.2: Sơ đồ Thực thể Liên kết (ERD) của hệ thống CSDL

Bảng 2.5: Đặc tả các Bảng thực thể trong Cơ sở dữ liệu

Tên Bảng	Khóa & Chỉ mục (Index)	Mô tả chi tiết và Vai trò cốt lõi
Users	PRIMARY KEY (id) UNIQUE (nickname)	Quản lý định danh người dùng. Bảo mật tuyệt đối bằng cách chỉ lưu trữ password_hash và salt. Tích hợp các cờ bảo vệ chống hành vi dò quét tự động (Brute-force): failed_attempts và locked_until.
Rooms	PRIMARY KEY (id) UNIQUE (room_name)	Lưu trữ siêu dữ liệu của phòng trò chuyện. Hỗ trợ cấp phát mật khẩu độc lập qua băm mật mã. Ràng buộc khóa ngoại creator_nick tới bảng Users kèm theo cơ chế xóa lan truyền (ON DELETE CASCADE).
RoomBans	PRIMARY KEY (id) idx_roombans	Bảng trung gian chuyên trách quản lý danh sách đen (Blacklist) của từng phòng độc lập. Đảm bảo tính duy nhất qua ràng buộc liên hợp UNIQUE(room_name, nickname).
ChatHistory	PRIMARY KEY (id) idx_chat_room_ts	Lưu trữ nội dung thông điệp trao đổi. Áp dụng chỉ mục liên hợp (Composite Index) trên cụm trường (room, timestamp) nhằm tăng tốc vượt trội các truy vấn khôi phục lịch sử trò chuyện.
AuditLog	PRIMARY KEY (id) idx_audit_ts	Quản lý sổ cái nhật ký kiểm toán bảo mật. Ứng dụng mô hình "chuỗi khối băm" (Blockchain-lite): Mỗi bản ghi chứa trường prev_hash tham chiếu trực tiếp đến mã băm của bản ghi tiền nhiệm, giúp vô hiệu hóa mọi nỗ lực can thiệp hoặc sửa đổi dữ liệu trái phép.

2.3.2 Luồng Xác thực và Quản lý Phiên (AuthManager)

1. Cơ sở lý thuyết về Bảo mật Định danh và Phiên giao dịch

Trong các hệ thống phân tán, cơ chế xác thực (Authentication) và quản trị phiên (Session Management) là chốt chặn an ninh đầu tiên. Để phòng chống các dạng tấn công dò quét mật khẩu (Brute-force) hay tấn công bằng từ điển băm (Rainbow Tables), mật khẩu thô (Plain-text) tuyệt đối không được lưu trữ. Kỹ thuật Kéo giãn khóa (Key Stretching) thông qua hàm dẫn xuất PBKDF2 (Password-Based Key Derivation Function 2) kết hợp với muối mã hóa ngẫu nhiên (Cryptographic Salt) được sử dụng để gia tăng chi phí điện toán của kẻ tấn công theo cấp số nhân.

Đối với quản lý phiên, việc truy vấn Cơ sở dữ liệu vật lý (Disk I/O) liên tục để kiểm tra trạng thái người dùng sẽ tạo ra "nút thắt cổ chai" (Bottleneck) về hiệu năng. Do đó, lý thuyết bảo mật hiện đại ưu tiên sử dụng mô hình Phiên không trạng thái (Stateless Session) dựa trên Chữ ký mã hóa (Cryptographic Signatures) như JWT (JSON Web Token). Bằng việc sử dụng mã xác thực thông điệp băm (HMAC), máy chủ có thể ủy quyền cho máy khách lưu trữ thông tin trạng thái mà vẫn đảm bảo tính toàn vẹn và chống giả mạo (Tamper-proof).

2. Triển khai Luồng Xác thực và Quản lý Phiên trong mã nguồn

Hệ thống hiện thực hóa các nguyên lý trên thông qua hai module lõi: AuthManager (Quản lý luồng xác thực DB) và SessionToken (Bộ sinh và xác minh JWT-like). Cơ chế được vận hành theo ba pha chính:

- **Pha 1: Khởi tạo định danh (Đăng ký/Đăng nhập):** Khi Client gửi gói tin MSG_CONNECT_REQUEST, AuthManager tiến hành trích xuất nickname. Trong tiến trình đăng ký, hệ thống gọi `SHA256Hash::generateSalt()` để tạo muối ngẫu nhiên, sau đó băm mật khẩu bằng hàm `SHA256Hash::pbkdf2()`. Nếu đăng nhập, hệ thống trích xuất salt từ DB và tính toán lại chuỗi băm để so sánh bằng hàm `CRYPTO_memcmp()` nhằm phòng tránh rò rỉ thông tin qua thời gian tính toán (Timing Attack).
- **Pha 2: Cấp phát Token (Stateless Session):** Ngay khi xác thực DB thành công, máy chủ sẽ lập tức chấm dứt các truy vấn Disk I/O cho người dùng này. Lớp SessionToken tiến hành đóng gói các thông điệp định danh (Claims) bao gồm: sub (nickname), role (Quyền hạn), exp (Thời hạn tồn tại tối đa SESSION_LIFETIME_SEC) và jti (Mã định danh Token duy nhất). Khối dữ liệu này được băm cùng một Khóa bí mật (Secret Key) bằng thuật toán HMAC-SHA256 để tạo thành chuỗi Token trả về cho Client. Hệ thống đồng thời theo

dõi số hiệu bộ mô tả tệp (File Descriptor - FD) đang gắn với Token này thông qua cấu trúc `active_sessions_`.

- **Pha 3: Khôi phục kết nối (Zero-roundtrip DB):** Đặc thù của mạng TCP là tính không ổn định, Client có thể bị rớt mạng và kết nối lại (Reconnect) qua một FD mới. Lúc này, Client không cần cung cấp mật khẩu, mà đính kèm chuỗi Token cũ vào gói `MSG_RECONNECT`. `AuthManager::reconnectWithToken()` chỉ cần tính toán lại chữ ký HMAC trên RAM, nếu hợp lệ và không nằm trong danh sách đen (`blacklist_`), hệ thống lập tức tái lập `ClientSession` với FD mới.
- **Pha 4: Thu hồi (Revocation) và Chống tấn công:** Nhược điểm lớn nhất của mô hình Stateless Token là không thể thu hồi tức thời. Hệ thống khắc phục điều này bằng cơ chế `SessionToken::revoke()` cho phép đưa mã `jti` vào danh sách đen lưu trên RAM tĩnh, ngăn chặn tấn công phát lại (Replay Attack) nếu Token bị lộ. Đồng thời, hàm `AuthManager::incrementFailedAttempts()` giới hạn số lần sai mật khẩu để bảo vệ tài khoản khỏi kỹ thuật Brute-force.

Hình 2.3: Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên (Auth & Reconnect)

2.4 Giao thức Mật mã và Tính bảo mật

Để đảm bảo nguyên tắc bảo mật thông điệp (Confidentiality), toàn bộ luồng giao tiếp giữa Client và Server đều được đặt dưới lớp bảo vệ của phương pháp Trao đổi khóa lai (Hybrid Cryptography). Phương pháp này giải quyết được điểm yếu của mã hóa bất đối xứng (chậm) và mã hóa đối xứng (khó phân phối khóa).

2.4.1 Luồng Trao đổi khóa (Key Exchange Workflow)

1. Cơ sở lý thuyết về Mật mã lai (Hybrid Cryptography)

Để giải quyết trọn vẹn bài toán bảo mật kênh truyền, lý thuyết mật mã hiện đại đề xuất sự kết hợp giữa Mật mã Bất đối xứng (Asymmetric Cryptography) và Mật mã Đối xứng (Symmetric Cryptography) nhằm tối ưu hóa cả hai khía cạnh: An toàn phân phối khóa và Hiệu năng mã hóa.

Mật mã Bất đối xứng (điển hình là thuật toán RSA) sử dụng một cặp khóa Công khai/Bí mật (Public/Private Key) để đảm bảo khóa giao tiếp có thể được truyền tải an toàn trên một kênh không tin cậy (Insecure Channel) mà không sợ rò rỉ. Tuy

nhien, RSA đòi hỏi tài nguyên tính toán cực kỳ lớn, không phù hợp để mã hóa dữ liệu theo thời gian thực. Ngược lại, Mật mã Đối xứng (điển hình là AES) sở hữu tốc độ mã hóa vượt trội, đáp ứng tốt cho lưu lượng mạng cao, nhưng lại gặp khó khăn ở khâu phân phối khóa an toàn ban đầu.

Hệ thống kết hợp cả hai mô hình này bằng cách dùng RSA-2048 để thiết lập một đường hầm an toàn ban đầu (Key Exchange), qua đó trao đổi một Khóa phiên Đối xứng (Session Key). Toàn bộ luồng dữ liệu sau đó sẽ được mã hóa bằng Khóa phiên này theo chế độ AES-GCM. Đây là chế độ Mã hóa có Xác thực (Authenticated Encryption with Associated Data - AEAD), không chỉ cung cấp tính Bảo mật (Confidentiality) mà còn tạo ra các Thẻ xác thực (Authentication Tags) nhằm bảo vệ tính Toàn vẹn (Integrity), ngăn chặn tuyệt đối các cuộc tấn công thay đổi nội dung gói tin (Man-in-the-Middle Attack).

2. Quy trình Bắt tay Mật mã trong mã nguồn (KeyExchange)

Trong hệ thống thực tế, quy trình trao đổi khóa được triển khai độc lập ngay trước khi bất kỳ giao tiếp nghiệp vụ nào diễn ra, được quản lý chặt chẽ bởi hai module CryptoEngine và KeyExchange. Quá trình gồm ba bước:

- **Bước 1: Chào hỏi và Cấp phát Khóa Công khai:** Ngay khi một kết nối TCP được thiết lập (`HandshakeState::WAITING_HELLO`), `CryptoEngine` trích xuất Khóa Công khai RSA-2048 đã được khởi tạo sẵn (`getPublicKeyPEM()`). Máy chủ đóng gói chuỗi PEM này cùng với một `server_nonce` ngẫu nhiên (sinh bằng `CSPRNG`) vào gói tin `MSG_CRYPTO_KEY_OFFER` và gửi xuống máy khách.
- **Bước 2: Sinh Khóa Phiên (Session Key):** Máy khách nhận Khóa Công khai, tự động sinh ngẫu nhiên một khóa AES 256-bit (32 bytes) và một `client_nonce` (16 bytes). Khối dữ liệu 48 bytes này được mã hóa bằng Khóa Công khai RSA của máy chủ và gửi ngược lại thông qua gói tin `MSG_CRYPTO_KEY_ACCEPT`.
- **Bước 3: Thiết lập Đường hầm Bảo mật:** Hàm `KeyExchange::handleKeyAccept()` tiến hành giải mã khối dữ liệu bằng Khóa Bí mật RSA. Hệ thống bóc tách 32 bytes khóa AES và lưu trữ cục bộ vào bảng băm `session_keys_` trong bộ nhớ của `CryptoEngine`, ánh xạ trực tiếp theo `fd` của kết nối. Các bản rõ của khóa trên RAM lập tức bị xóa bỏ an toàn bằng lệnh `OPENSSL_cleanse()` nhằm tránh rò rỉ bộ nhớ.
- **Cơ chế phòng thủ chủ động:** Nếu quá trình giải mã thất bại, sai kích thước blob, hoặc vượt quá thời gian cho phép (`HANDSHAKE_TIMEOUT_SEC`), `KeyExchange` lập tức đánh dấu `HandshakeState::FAILED` và cắt đứt kết nối. Đồng thời, hệ thống báo cáo địa chỉ IP vi phạm cho `IntrusionDetector` để ghi nhận

các hành vi tấn công Dò quét cổng (Port Scan) hoặc Phá vỡ mã hóa (Cryptocracking), từ đó áp dụng lệnh cấm IP (IP_BLOCK_SECONDS) nếu phát hiện tần suất vi phạm bất thường.

Hình 2.4: Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES

2.5 Hệ thống Chức năng Cốt lõi

2.5.1 Luồng dữ liệu Trò chuyện và Quản lý phòng (RoomManager)

1. Cơ sở lý thuyết về Định tuyến Thông điệp và Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe)

Trong kiến trúc phần mềm phân tán đa người dùng (Multi-user Distributed Systems), việc quản lý và phân phối thông điệp thời gian thực đặt ra thách thức lớn về sự tranh chấp tài nguyên và hiệu năng định tuyến. Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe Pattern) được ứng dụng rộng rãi như một mô hình lý thuyết tiêu chuẩn để giải quyết bài toán này. Trong đó, mỗi "Phòng trò chuyện" (Chat Room) đóng vai trò là một Chủ đề (Topic/Channel). Người gửi (Publisher) đẩy thông điệp vào kênh, và hệ thống sẽ tự động sao chép, phân phối tới tất cả các thành viên đang theo dõi (Subscribers) kênh đó.

Đồng thời, để đảm bảo tính Nhất quán dữ liệu (Data Consistency) khi hàng ngàn luồng truy cập đồng thời vào cấu trúc quản lý kênh, hệ thống áp dụng cơ chế Khóa Đọc-Ghi (Reader-Writer Lock). Cơ chế này cho phép đa luồng đọc (Broadcast) song song mà không bị nghẽn, nhưng độc quyền hóa luồng ghi (Join/Leave/Create) để chống hiện tượng tranh bộ nhớ (Race Condition). Bên cạnh đó, việc thanh lọc chuỗi (String Sanitization) là bắt buộc ở cửa ngõ đầu vào nhằm ngăn chặn các lỗ hổng tiêm nhiễm mã độc (XSS, Injection).

2. Triển khai Định tuyến và Quản lý phòng trong mã nguồn

Hệ thống hiện thực hóa mô hình Pub/Sub thông qua lớp lõi RoomManager. Luồng dữ liệu trò chuyện MSG_CHAT_SEND được xử lý theo ba tầng kiến trúc:

- **Tầng 1: Thanh lọc Dữ liệu (Sanitization):** Ngay khi tiếp nhận Payload, chuỗi văn bản message bị ép phải đi qua module MessageFilter. Bằng các biểu thức chính quy (Regex), mọi ký tự có nguy cơ thực thi mã độc như thẻ `<script>`, nháy đơn ('), nháy kép (") đều bị mã hóa thành văn bản thuần. Đây là chốt chặn bảo mật phi trạng thái (Stateless).

- **Tầng 2: Định tuyến và Khuếch đại (Routing & Broadcast):** Hệ thống phân tách luồng xử lý:
 - *Direct Message (Cá nhân):* Tra cứu `ClientSession` của người nhận qua số hiệu bộ mô tả tệp (`fd`) và chuyển tiếp nguyên bản gói tin JSON.
 - *Room Message (Phòng nhóm):* Hàm `RoomManager::broadcastToRoom()` được kích hoạt. Hàm này cấp phát một khóa đọc `std::shared_lock<std::shared_m` để truy xuất danh sách `members` (`Subscribers`) của `Room`. Vòng lặp sẽ sao chép và phân phối thông điệp tới từng `fd` thông qua `ClientSession::sendPacket()`, đồng thời loại trừ `excludeFd` (Người gửi).
- **Tầng 3: Lưu trữ Lịch sử và Đồng bộ (Persistence):** Mỗi `Room` được liên kết động 1-1 với một đối tượng `ChatHistory`. Hàm `RoomManager::appendHistory()` kết xuất tin nhắn xuống SQLite theo nguyên lý Ghi trước Nhật ký (WAL). Khi một `Client` mới tham gia phòng (`joinRoom()`), hàm `sendHistoryToClient()` lập tức truy xuất N tin nhắn gần nhất và đẩy xuống `Client`, đảm bảo tính liên tục của ngữ cảnh giao tiếp.
- **Cơ chế Quản trị Vòng đời (Lifecycle Management):** Bảng 2.6 minh họa chiến lược quản trị đồng thời (Concurrency Strategy) được áp dụng trên biến `m_mutex` của `RoomManager`. Đặc biệt, hệ thống tích hợp thuật toán Thu gom rác tự động (Garbage Collection): nếu hàm `leaveRoom()` phát hiện `getMemberCount() == 0`, nó sẽ tự động giải phóng đối tượng `Room` và `ChatHistory` khỏi RAM nhằm tối ưu hóa tài nguyên.

Bảng 2.6: Chiến lược cấp phát khóa (Locking) trong `RoomManager`

Tên Hàm (Method)	Loại Khóa (Lock Type)	Mục đích Kỹ thuật
<code>broadcastToRoom()</code> <code>getUsersInRoom()</code> <code>getRoomList()</code>	<code>std::shared_lock</code> (Khóa Đọc)	Cho phép luồng phân phối tin nhắn và truy xuất danh sách hoạt động đa nhiệm song song không giới hạn, không gây nghẽn cổ chai (Non-blocking).

Tên Hàm (Method)	Loại Khóa (Lock Type)	Mục đích Kỹ thuật
createRoom() deleteRoom()	std::unique_lock (Khóa Ghi)	Độc quyền thay đổi cấu trúc bảng băm m_rooms và m_histories. Đảm bảo tính Toàn vẹn cấu trúc (Structural Integrity).
joinRoom() leaveRoom() removeUser()	std::unique_lock (Khóa Ghi)	Ngăn ngừa tương tranh cục bộ khi cập nhật danh sách members của phòng và con trỏ ngược m_fdToRoom.

Hình 2.5: Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng (RoomManager)

2.5.2 Trung chuyển Tập tin ẩn danh (Server-Relayed File Transfer)

1. Cơ sở lý thuyết về Kiến trúc Trung chuyển Ẩn danh

Trong các giao thức chia sẻ tập tin thông thường như P2P (Peer-to-Peer), hai thiết bị đầu cuối phải thiết lập kết nối trực tiếp, dẫn đến việc lộ lọt địa chỉ IP công cộng. Điều này tạo ra rủi ro nghiêm trọng về quyền riêng tư và mở ra bề mặt tấn công cho các chiến dịch từ chối dịch vụ (DDoS) hoặc truy vết danh tính (Doxing).

Để khắc phục, hệ thống áp dụng mô hình Trung chuyển qua Máy chủ (Server-Relayed Architecture) kết hợp Phễu Proxy (Proxy Funneling). Máy chủ đóng vai trò là "Người đứng giữa hợp pháp", tiếp nhận các đoạn dữ liệu (Chunks) từ người gửi và định tuyến lại cho người nhận. Mô hình này đảm bảo ẩn danh hoàn toàn (Anonymity) do cả hai bên chỉ biết đến IP của Server. Đồng thời, thay vì lưu tập tin xuống ổ cứng vật lý để chuyển tiếp (Store-and-Forward), hệ thống sử dụng Kỹ thuật Đệm bộ nhớ tạm thời (In-Memory Buffering/Zero-Disk I/O) giúp triệt tiêu hoàn toàn độ trễ đọc/ghi đĩa và tránh nguy cơ tràn dung lượng máy chủ do rác dữ liệu.

2. Triển khai Luồng Truyền tập tin trong mã nguồn

Tầng xử lý tập tin được điều phối bởi lớp tĩnh FileTransfer. Vòng đời của một phiên truyền tập tin được vận hành như một Máy trạng thái hữu hạn (Finite State Machine - FSM) với 4 pha cốt lõi:

- **Pha 1: Yêu cầu và Xác thực (PENDING):** Khi người gửi phát gói MSG_FILE_REQUEST, hệ thống sinh ra một tid (Transfer ID) bằng bộ định danh CSPRNG. Hàm `isAllowedExtension()` thực hiện kiểm duyệt an ninh gắt gao (Whitelist) chỉ cho phép .txt, .png, .jpg, .pdf. Dung lượng bị giới hạn chặt ở mức 3MB. Nếu hợp lệ, hệ thống khởi tạo đối tượng `FileTransferSession` lưu trên bảng băm RAM.
- **Pha 2: Chấp thuận (ACCEPTED):** Người nhận được máy chủ thông báo và có quyền từ chối hoặc chấp nhận. Nếu từ chối qua gói MSG_FILE_REJECT, Session lập tức bị hủy bỏ. Nếu chấp nhận qua gói MSG_FILE_ACCEPT, trạng thái chuyển sang ACCEPTED, mở khóa cho luồng dữ liệu tiếp theo.
- **Pha 3: Luồng Chuyển tiếp (IN_PROGRESS):** Người gửi băm nhỏ tệp thành các phần (Chunks) và gửi liên tục qua gói MSG_FILE_DATA. Hàm `FileTransfer::handle` kiểm tra quyền sở hữu `senderFd` (Security Check) để chống giả mạo, sau đó ngay lập tức chuyển tiếp (Relay) payload này tới `receiverFd`. Dữ liệu chỉ nằm trên RAM trong vài mili-giây trước khi được đẩy xuống socket đích.
- **Pha 4: Hoàn tất và Dọn dẹp rác (COMPLETE / Garbage Collection):** Khi hoàn tất, gói MSG_FILE_COMPLETE kích hoạt việc ghi nhận Kiểm toán (AUDIT_D) và xóa Session. Đặc biệt, để xử lý các phiên bị treo do mất kết nối (Zombies), bộ thu gom rác `cleanupExpired()` được Event Loop gọi định kỳ để tiêu hủy mọi Session vượt quá ngưỡng `TIMEOUT_SEC`.

Bảng 2.7 tổng hợp các Trạng thái hợp lệ (Valid States) trong vòng đời của `FileTransferSession`Bảng 2.7: Các trạng thái của Phiên truyền tệp
(`FileTransferSession`)

Trạng thái (State)	Gói tin Kích hoạt	Mô tả Nghiệp vụ
PENDING	MSG_FILE_REQUEST	Chờ người nhận xác nhận. Khóa chức năng truyền luồng dữ liệu.
ACCEPTED	MSG_FILE_ACCEPT	Người nhận đồng ý. Mở khóa cho phép tiếp nhận gói Data đầu tiên.
IN_PROGRESS	MSG_FILE_DATA	Đang trong quá trình trung chuyển các đoạn dữ liệu (Chunks).

Trạng thái (State)	Gói tin Kích hoạt	Mô tả Nghiệp vụ
(Destroyed)	MSG_FILE_COMPLETE	Session bị xóa khỏi bộ nhớ
	MSG_FILE_REJECT	m_transfers. Hoàn tất vòng đời.

Hình 2.6: Máy trạng thái hữu hạn (FSM) luồng Truyền tệp ẩn danh

2.6 Hệ thống Quản trị và Chống lạm dụng

2.6.1 Hàng rào Chống Spam và DDoS (Tường lửa mềm Application-Layer)

1. Cơ sở lý thuyết về Phòng chống Tấn công Từ chối Dịch vụ (DoS/DDoS)

Bất kỳ máy chủ TCP nào khi mở cổng giao tiếp công khai trên Internet đều phải đối mặt với nguy cơ bị dội bom yêu cầu (Flooding). Các cuộc tấn công Từ chối Dịch vụ Phân tán (DDoS) ở Tầng 7 (Application Layer) thường gửi một lượng lớn các thông điệp hợp lệ nhằm vắt kiệt tài nguyên xử lý (CPU) và bộ nhớ (RAM) của máy chủ.

Thay vì phụ thuộc hoàn toàn vào các Tường lửa vật lý hay Reverse Proxy (như NGINX), lý thuyết Bảo mật Chiều sâu (Defense in Depth) khuyến nghị tích hợp ngay một "Tường lửa mềm" vào trong hệ thống. Phương pháp cốt lõi được sử dụng là thuật toán Giỏ thẻ bài (Token Bucket). Mô hình này cấp cho mỗi thực thể một số lượng thẻ (Tokens) nhất định. Mỗi thao tác sẽ tiêu tốn một thẻ. Giỏ sẽ được nạp lại theo một Tốc độ cố định (Refill Rate). Khác biệt với việc chỉ thả gói (Packet Dropping), lý thuyết bảo mật hiện đại yêu cầu kết hợp Hệ thống Phát hiện Xâm nhập (IDS) để phân tích hành vi và chủ động đánh giá mức độ rủi ro (Threat Scoring) nhằm cấm vĩnh viễn các tác nhân độc hại.

2. Triển khai Hệ thống RateLimiter và IntrusionDetector trong mã nguồn

Hệ thống phòng thủ đa tầng này được điều phối bởi hai module độc lập là RateLimiter và IntrusionDetector:

- **Tầng 1: Bộ giới hạn tốc độ (RateLimiter - Token Bucket):** Hàm `TokenBucket::consume` được thực thi liên tục để trừ điểm Token dựa trên độ trễ thời gian (`elapsed * refill_rate`). Các ngưỡng giới hạn được áp dụng chi tiết tùy loại tác vụ như MSG_CHAT, CONNECT, AUTH. Nếu Client tiêu thụ hết Token và vi phạm quá giới

hạn (`violations >= 4`), truy vấn bị từ chối (`RATE_LIMITED`) nhằm bảo vệ CPU khỏi việc phải giải mã các JSON rác.

- **Tầng 2: Hệ thống chấm điểm Rủi ro (IntrusionDetector):** Mọi vi phạm từ `RateLimiter` hay các module khác đều được gom về hàm `reportViolation()`. Tại đây, mỗi lỗi được quy ra một điểm số (`Threat Score`) tùy theo mức độ nghiêm trọng. Ví dụ: Nhấn rác bị phạt 5 điểm, trong khi phát hiện Quét cổng (`Port Scan`) bị phạt tới 100 điểm.
- **Tầng 3: Biện pháp Trừng phạt lũy tiến (Progressive Penalty):** Hàm `checkThresholds()` phân tích tổng điểm hiện tại để đưa ra quyết định:
 - Đạt mốc 100 điểm (`THREAT_THRESHOLD`): Cấm IP tạm thời 1 giờ (`m_tempBans`).
 - Đạt mốc 200 điểm (`ATTACK_THRESHOLD`): Cấm IP tạm thời 24 giờ.
 - Chạm mốc 500 điểm (`PERM_BAN_THRESHOLD`): Hệ thống ghi log Kiểm toán (`AuditLogger`), gọi `exportBanList()`, và đẩy IP vào tập hợp `m_permBans` để cấm vĩnh viễn ở cấp độ Network, loại bỏ tận gốc mối đe dọa.

Bảng 2.8 minh họa hệ thống chấm điểm phạt (`Threat Scoring`) được lập trình tĩnh trong `IntrusionDetector::scoreForViolation()`.

Bảng 2.8: Hệ thống Điểm phạt Hành vi (`Threat Scoring System`)

Hành vi Vi phạm (Violation Type)	Điểm phạt	Mô tả Nghiệp vụ
<code>RATE_LIMIT_EXCEEDED</code>	5	Spam tin nhắn/Gửi request quá tần suất cho phép của Giỏ thẻ bài.
<code>FAILED_AUTH</code> <code>INVALID_PACKET</code>	10	Cố gắng dò mật khẩu thất bại hoặc gửi cấu trúc JSON bị dị hình.
<code>HMAC_FAILURE</code>	20	Sai chữ ký điện tử (Nghỉ ngờ cố ý giả mạo Token).
<code>INJECTION_ATTEMPT</code>	30	Bị <code>MessageFilter</code> phát hiện chèn mã độc XSS/SQLi.
<code>REPLAY_ATTACK</code>	50	Tấn công phát lại (Sử dụng lại chuỗi <code>jti</code> đã bị thu hồi).

Hành vi Vi phạm (Violation Type)	Điểm phạt	Mô tả Nghiệp vụ
PORT_SCAN	100	Bị Timeout khi bắt tay mã hóa lại, dấu hiệu dò quét công mạng.

Hình 2.7: Lược đồ Khối (Block Diagram) của hệ thống Tường lửa mềm (RateLimiter & IDS)

2.6.2 Phân quyền RBAC và Quản trị

1. Cơ sở lý thuyết về Kiểm soát Truy cập dựa trên Vai trò (RBAC)

Trong quản trị hệ thống tập trung, việc cấp phát quyền (Permissions) trực tiếp cho từng cá nhân sẽ dẫn đến sự bùng nổ độ phức tạp khi quy mô người dùng tăng lên (Role Explosion). Để chuẩn hóa, mô hình Kiểm soát Truy cập dựa trên Vai trò (Role-Based Access Control - RBAC) được áp dụng.

RBAC hoạt động bằng cách nhóm các quyền hạn vào những Vai trò (Roles) cụ thể. Người dùng không giữ quyền, mà được gán vào các Vai trò này. Đồng thời, hệ thống tuân thủ nghiêm ngặt Nguyên tắc Đặc quyền tối thiểu (Principle of Least Privilege), trong đó tài khoản mới tạo luôn bắt đầu ở Vai trò thấp nhất. Hơn nữa, để đảm bảo tính Trách nhiệm giải trình (Accountability), mọi hành động thay đổi trạng thái của các Vai trò có đặc quyền (Privileged Actions) đều bắt buộc phải được ghi nhận lại trong Nhật ký Kiểm toán (Audit Trails) để ngăn chặn việc lạm quyền.

2. Triển khai Phân quyền trong mã nguồn (AdminCommands)

Hệ thống phân cấp quyền hạn thành 3 cấp bậc rõ ràng (USER, ADMIN, OWNER), được vận hành và bảo vệ bởi lớp AdminCommands. Quá trình kiểm duyệt quyền được tập trung hóa qua hàm `hasAdminRight(adminFd, ownerOnly)`, đảm bảo không có lỗ hổng leo thang đặc quyền (Privilege Escalation).

- **USER (Vai trò mặc định):** Chỉ có quyền giao tiếp cơ bản (Nhắn tin, Truyền tệp). Bị giám sát chặt chẽ bởi RateLimiter và IntrusionDetector.
- **ADMIN (Quản trị viên Hệ thống):** Cấp bậc vận hành, có khả năng thực thi các lệnh cưỡng chế:
 - `/mute & /unmute`: Khóa hoặc mở khóa tính năng gửi tin nhắn của một Session cụ thể.

- /kick & /unkick: Trục xuất người dùng khỏi phòng chat thông qua lệnh `m_server->kickFromRoom()`.
- /ban & /unban: Tương tác trực tiếp với `IntrusionDetector::permBan()` để khóa vĩnh viễn IP và Nickname của đối tượng vi phạm.
- /broadcast: Phóng thanh thông báo hệ thống (System Notification) đến toàn bộ các phòng.
- **OWNER (Chủ sở hữu Tối cao):** Là vai trò duy nhất thỏa mãn tham số `ownerOnly = true`. OWNER có độc quyền sử dụng lệnh /promote (Thăng cấp một USER lên ADMIN) và /demote (Giáng cấp ADMIN xuống USER).
- **Kiểm toán Đặc quyền (Audit Logging):** Bất kỳ lệnh quản trị nào khi thực thi thành công đều sẽ gọi macro `AUDIT_D()` để kết xuất log qua lớp `AuditLogger`. Ví dụ, khi OWNER dùng lệnh /kick, hệ thống sẽ ghi lại `AuditEventType::ADMIN`, tên người đá, tên người bị đá, và lý do (reason) xuống tệp bảo mật, đáp ứng chuẩn quy trình thanh tra hệ thống.

Bảng 2.9 tổng hợp Ma trận Phân quyền (Access Control Matrix) thực tế trong hệ thống.

Bảng 2.9: Ma trận Phân quyền (RBAC Matrix) của Hệ thống

Tác vụ (Command)	USER	ADMIN	OWNER
Nhắn tin / Chuyển tệp / Đổi tên	Có	Có	Có
/mute / /kick / /broadcast	Không	Có	Có
/ban / /unban	Không	Có	Có
/promote / /demote	Không	Không	Có

CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này đi sâu vào quá trình thiết lập môi trường hoạt động thực tế, quy trình biên dịch sản phẩm cũng như trình bày các kết quả đánh giá kỹ thuật (Benchmarks) xoay quanh giới hạn chịu tải, độ trễ và khả năng chống chịu mã độc/Spam của phần mềm.

3.1 Môi trường và Công nghệ Triển khai

3.1.1 Cấu hình Môi trường Hệ thống

1. Cơ sở lý thuyết về Tương thích Nền tảng (Platform Compatibility) và I/O Phi đồng bộ

Trong phát triển phần mềm mạng hiệu năng cao, cơ chế Vào/Ra (I/O) quyết định giới hạn chịu tải của hệ thống. Các hệ điều hành lõi Linux/Unix cung cấp API `epoll` với độ phức tạp $O(1)$, cho phép theo dõi hàng chục nghìn Socket đồng thời mà không bị thắt nút cổ chai (Bottleneck) như hàm `select()` $O(N)$ cổ điển. Ngược lại, nền tảng Windows sử dụng kiến trúc hoàn toàn khác là IOCP (I/O Completion Ports). Do sự phân mảnh API này, các máy chủ phân tán (Distributed Servers) thường được ưu tiên triển khai độc quyền trên Linux để tối ưu hóa hiệu năng và độ ổn định.

Đối với quy trình Xây dựng (Build Process), lý thuyết Quản trị Cấu hình (Configuration Management) khuyến nghị sử dụng các Hệ thống Siêu biên dịch (Meta-Build Systems) như CMake. Thay vì viết cứng các tệp `Makefile` phụ thuộc vào một hệ điều hành cụ thể, CMake cung cấp lớp trừu tượng (Abstraction Layer) để tự động sinh ra kịch bản biên dịch bản địa (Native Build Scripts) tương thích chéo nền tảng.

2. Kiến trúc Biên dịch trong tệp `CMakeLists.txt`

Hệ thống được thiết kế thành một Monorepo (Kho lưu trữ mã nguồn tập trung) quản lý chung cả `vcs_server` và `vcs_client`. Việc phân mảnh nền tảng được định tuyến chặt chẽ thông qua các cờ CMake:

- **Máy chủ (Server-Side):** Bị khóa chặt (Vendor Lock-in) vào môi trường POSIX (Ubuntu/CentOS). Lệnh `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))` trong cấu hình CMake sẽ ngăn chặn hoàn toàn việc build Server trên Windows nhằm đảm bảo module `epoll` hoạt động chuẩn xác và an toàn nhất.
- **Máy khách (Client-Side):** Được thiết kế đa nền tảng (Cross-platform). Nếu biến `WIN32` được kích hoạt, CMake sẽ tự động tiến hành liên kết tĩnh (Static

Linking) với các thư viện lỗi của Microsoft gồm `wsock32`, `ws2_32` (Winsock API cho mạng) và `crypt32` (dành cho chứng chỉ bảo mật).

- **Quản lý Phụ thuộc (Dependency Injection):** Dự án ép buộc chuẩn C++17 (`CMAKE_CXX_STANDARD 17`) cùng các cờ tối ưu hóa cấp độ cao `-Wall -Wextra -O2`. Khâu đóng gói yêu cầu 4 thư viện lỗi: **OpenSSL** (`libssl-dev`) cho mật mã lai, **SQLite3** (`libsqlite3-dev`) cho lưu trữ bền vững, **Threads** (POSIX Threads) cho xử lý đa luồng, và **nlohmann/json** cho phân tích Payload.

Bảng 3.1 liệt kê chi tiết các thành phần phụ thuộc (Dependencies) và vai trò của chúng trong biểu đồ kiến trúc hệ thống.

Bảng 3.1: Bảng Ma trận Phụ thuộc Thư viện (Dependency Matrix)

Tên Thư viện	Loại Liên kết	Mục đích Kỹ thuật
OpenSSL::SSL OpenSSL::Crypto	Động (Dynamic)	Khởi tạo môi trường Bất tay Mật mã Lai (RSA) và mã hóa luồng khối (AES-GCM). Cung cấp hàm sinh số ngẫu nhiên an toàn CSPRNG.
SQLite3	Động (Dynamic)	Hệ quản trị cơ sở dữ liệu quan hệ cục bộ (Local RDBMS) phục vụ module AuthManager và lưu trữ ChatHistory.
Threads::Threads	Hệ thống (System)	Khai thác POSIX Threads (pthreads) để vận hành các Worker Threads và vòng lặp Event Loop song song.
nlohmann/json	Header-only	Cung cấp bộ tuần tự hóa/giải tuần tự hóa (Serialize/Deserialize) các chuỗi JSON payload phi cấu trúc.

Tên Thư viện	Loại Liên kết	Mục đích Kỹ thuật
wsock32, ws2_32	Tĩnh (Static) (Chỉ Windows)	Cung cấp API Windows Sockets để khởi tạo kết nối TCP thay thế cho Socket POSIX truyền thống ở phía Client.

3.1.2 Kịch bản biên dịch và vận hành (Build & Execution Pipeline)

1. Cơ sở lý thuyết về quy trình biên dịch và hệ thống vận hành

Trong kỹ thuật phần mềm hiện đại, quy trình xây dựng (Build Process) đóng vai trò cốt lõi trong việc đảm bảo tính nhất quán, khả năng tái sử dụng và độ tin cậy của mã nguồn khi triển khai đa nền tảng. Hệ thống sử dụng **CMake** làm công cụ quản trị vòng đời biên dịch (Meta-build System). Điểm mạnh của CMake là khả năng trừu tượng hóa môi trường (Environment Abstraction), cho phép sinh ra các kịch bản biên dịch bản địa (Native Build Scripts) tương thích với nhiều hệ điều hành và trình biên dịch khác nhau.

Bên cạnh đó, dự án ứng dụng mẫu thiết kế **Biên dịch ngoại tuyến (Out-of-Source Build)**. Cụ thể, thay vì đặt các tệp đối tượng trung gian (như .o, .so) trực tiếp vào cấu trúc mã nguồn, toàn bộ quá trình đóng gói được cô lập tại thư mục build/. Việc này duy trì tính tinh gọn của hệ thống quản lý phiên bản (Version Control System), đồng thời cho phép làm sạch không gian biên dịch (Clean Workspace) một cách tuyệt đối mà không gây rủi ro thất thoát dữ liệu.

Đối với quá trình vận hành, hệ thống tận dụng kỹ thuật **Biên dịch song song (Parallel Compilation)** thông qua cơ chế Đa xử lý đối xứng (SMP) của CPU. Bằng cách chia nhỏ các tác vụ biên dịch độc lập và thực thi đồng thời trên nhiều luồng xử lý tương ứng với số nhân vật lý, thời gian đóng gói phần mềm được tối ưu hóa đáng kể.

2. Kịch bản biên dịch trong thực tế hệ thống

Kiến trúc biên dịch thực tế của dự án được kiểm soát thông qua tệp cấu hình `CMakeLists.txt` và kịch bản đóng gói `build_all.sh`. Cấu hình yêu cầu chuẩn C++17 kết hợp cùng các cờ tối ưu hóa `-Wall -Wextra -O2` nhằm kiểm duyệt lỗi tĩnh khắt khe và tăng cường hiệu suất mã nhị phân.

Quá trình biên dịch được thiết kế thành hai phân luồng biệt lập:

- **Phân luồng Máy chủ (Server-side):** Giới hạn biên dịch hoàn toàn trên môi trường lõi POSIX (Linux). Hệ thống khóa nền tảng (Vendor Lock-in) máy chủ thông qua cờ kiểm tra `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))`. Trình biên dịch sẽ liên kết trực tiếp với các thư viện lõi POSIX Threads, OpenSSL và hệ quản trị cơ sở dữ liệu SQLITE3.
- **Phân luồng Máy khách (Client-side):** Hỗ trợ biên dịch chéo nền tảng (Cross-platform Compilation). Đối với môi trường Windows, tệp `mingw-toolchain.cmake` được sử dụng để định tuyến cấu trúc MinGW và liên kết tĩnh các thư viện mạng đặc thù của Microsoft như `wsock32`, `ws2_32` cùng `crypt32`.

Bảng 3.2 trình bày chi tiết các tập lệnh thực thi trong vòng đời biên dịch thực tế.

Bảng 3.2: Các tập lệnh và tham số cấu hình trong chu trình biên dịch

Giai đoạn	Tập lệnh (Command)	Mục đích kỹ thuật
1. Khởi tạo Không gian	<code>mkdir -p build/linux &&</code> <code>cd build/linux</code>	Thiết lập môi trường biên dịch ngoại tuyến (Out-of-source Build) cho Linux.
2. Thiết lập CMake	<code>cmake ../..</code>	Phân tích <code>CMakeLists.txt</code> , định tuyến thư viện và cấu trúc Makefile bản địa.
3. Biên dịch Đa luồng	<code>make -j\$(nproc)</code>	Kích hoạt biên dịch song song để giảm thiểu thời gian đóng gói nhị phân.
4. Biên dịch Chéo Windows	<code>cmake</code> <code>-DCMAKE_TOOLCHAIN_FILE=...</code> <code>make -j\$(nproc)</code> <code>vcs_client</code>	Triển khai biên dịch chéo ứng dụng Client cho nền tảng Windows trên máy chủ Linux.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_BIEN_DICH]

3. Chu trình vận hành và tham số dòng lệnh (Execution Lifecycle)

Sau khi hệ thống được đóng gói thành công, các tệp thực thi cuối cùng được nhúng hệ thống diễn dịch tham số dòng lệnh (Command-line Arguments). Cơ chế này cho phép quản trị viên cấp phát cấu hình linh hoạt trong thời gian chạy (Runtime) mà không đòi hỏi tái biên dịch mã nguồn.

1. Phân hệ vận hành Máy chủ (Server Execution)

Tệp thực thi `vcs_server` chịu trách nhiệm khởi tạo kiến trúc đa luồng, cấp phát cơ sở dữ liệu và vận hành hệ thống Socket lắng nghe. Khi được khởi chạy, luồng `main()` sẽ tự động thiết lập thư mục `logs/` với phân quyền `0755` và khởi tạo cơ sở dữ liệu `vcs_chat.db`.

Bảng 3.3 mô tả chi tiết các tham số khởi động định tuyến của hệ thống máy chủ.

Bảng 3.3: Các tham số dòng lệnh vận hành Máy chủ

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
<code>--config <path></code>	Chuỗi (String)	Nạp tệp cấu hình JSON khai báo các thông số như quy mô luồng (Thread Pool) và số lượng tối đa kết nối (Mặc định sử dụng <code>server_config.json</code>).
<code>--port <port_number></code>	Số nguyên (Int)	Ghi đè cổng dịch vụ mạng (Network Port), thay thế thông số tương ứng trong tệp cấu hình JSON.
<code>--verify-audit-log</code>	Cờ (Boolean)	Kích hoạt phương thức <code>AuditLogger::verifyChain()</code> nhằm kiểm tra tính toàn vẹn của chuỗi Nhật ký Kiểm toán. Tiến trình khởi động sẽ bị ngắt nếu phát hiện sự bất thường.

2. Phân hệ vận hành Máy khách (Client Execution)

Ứng dụng `vcs_client` xử lý việc thiết lập luồng giao tiếp bảo mật với máy chủ và cung cấp vòng lặp sự kiện tương tác người dùng thông qua `ConnectionManager`. Đối

với hệ điều hành lõi Windows, trong giai đoạn khởi tạo (Initialization), trình khách sẽ tự động thực thi hàm WSStartup để tải cấu trúc thư viện winsock2 trước khi cấp phát Socket mạng.

Bảng 3.4 thống kê cấu trúc tham số định tuyến hỗ trợ cho quá trình kết nối của máy khách.

Bảng 3.4: Các tham số dòng lệnh kết nối Máy khách

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
--host <ip_address>	Chuỗi (String)	Định tuyến địa chỉ IPv4/IPv6 của máy chủ đích (Mặc định được khởi tạo tại vòng lặp cục bộ 127.0.0.1).
--port <port_number>	Số nguyên (Int)	Chỉ định cổng mạng giao tiếp của máy chủ đích (Mặc định thiết lập tại cổng 9000).

[PLACEHOLDER_HINH_ANH_CHU_TRINH_VAN_HANH]

3.2 Kịch bản Kiểm thử và Đánh giá Hiệu năng

3.2.1 Tổng quan hệ thống Kiểm thử tự động (Test Framework)

1. Cơ sở lý thuyết về phương pháp luận kiểm thử

Trong chu trình phát triển phần mềm hiện đại, Phương pháp Phát triển Hướng Kiểm thử (Test-Driven Development - TDD) và hệ thống Tích hợp Liên tục (Continuous Integration - CI) đóng vai trò then chốt trong việc duy trì độ ổn định và chất lượng mã nguồn. Thay vì triển khai kiểm thử thủ công (Manual Testing) vốn tiêu tốn tài nguyên và dễ phát sinh sai sót, hệ thống được trang bị một hệ sinh thái kiểm thử tự động (Automated Testing Framework) tích hợp trực tiếp vào quy trình đóng gói.

Hệ thống được thiết kế bao gồm hai cấp độ kiểm thử chuyên sâu:

- **Kiểm thử Đơn vị (Unit Testing):** Được thực thi tại mức mã nguồn (Code-level) nhằm cô lập và xác thực tính đúng đắn của từng hàm hoặc thành phần (Module) cụ thể. Kỹ thuật này cho phép phát hiện sớm các lỗi hỏng logic từ giai đoạn phát triển

ban đầu (Shift-left Testing).

- **Kiểm thử Tích hợp (Integration/E2E Testing):** Xác minh luồng giao tiếp dữ liệu liên thông giữa ứng dụng máy khách và máy chủ, mô phỏng các kịch bản tương tác thực tế dựa trên phương pháp hộp đen (Black-box Testing).

2. Cấu trúc Test Framework trong thực tế mã nguồn

Thay vì phụ thuộc vào các thư viện bên thứ ba đồ sộ (như Google Test hay Catch2) làm gia tăng chi phí biên dịch, dự án đã tự chủ thiết kế một vi khung kiểm thử (Micro Test Framework) bằng C++ thuần túy thông qua các tệp nền tảng `TestMacro.h` và `test_framework.h`. Hệ thống này cung cấp các hàm Macro tiêu chuẩn hóa như `ASSERT_TRUE`, `ASSERT_EQ`, và `ASSERT_THROWS` để đánh giá các biểu thức logic tĩnh.

Thông qua chỉ thị `enable_testing()` được khai báo trong `CMakeLists.txt`, hệ thống đăng ký và vận hành tự động một tập hợp các kịch bản kiểm thử (Test Suites) độc lập:

- **Khối Kiểm thử Mật mã (`test_crypto`):** Đo lường độ phức tạp thời gian thực thi (Time Complexity) và xác thực tính đồng nhất của dữ liệu sau quá trình mã hóa và giải mã lai giữa phương thức bất đối xứng RSA-2048 và luồng khối AES-256.
- **Khối Kiểm thử Xác thực (`test_auth` & `test_auth_manager`):** Kiểm định cơ chế băm mật khẩu (Password Hashing) sử dụng thuật toán PBKDF2 kết hợp giá trị nhiễu (Salting), đồng thời kiểm tra sự nhất quán trong luồng ghi cơ sở dữ liệu nội bộ SQLite3.
- **Khối Kiểm thử Giao thức (`test_protocol`):** Xác thực cơ chế Tuần tự hóa (Serialization) và Phân tích cú pháp (Parsing) của giao thức cấu trúc gói tin nhị phân. Các bài kiểm tra đặc biệt tập trung vào khả năng bắt ngoại lệ (Error Handling) khi đối mặt với các cấu trúc JSON bị biến dạng (Malformed JSON).
- **Khối Kiểm thử Giới hạn Dòng (`test_ratelimit`):** Mô phỏng các xung lưu lượng mạng (Traffic Spikes) đột biến nhằm kiểm chứng năng lực phản xạ, từ chối dịch vụ của thuật toán Token Bucket tại biên giới mạng.

Bên cạnh các bài kiểm thử C++, dự án tích hợp hệ thống kiểm thử tự động hộp đen viết bằng ngôn ngữ Python (`integration_test.py`). Kịch bản này tương tác trực tiếp qua luồng TCP (Raw Socket) nhằm kiểm chứng sự bền bỉ của kiến trúc đa luồng trước các trạng thái dị thường từ máy khách.

Bảng 3.5 trình bày ma trận phân bổ các module kiểm thử tự động của hệ thống.

Bảng 3.5: Ma trận phân bổ các module kiểm thử tự động

Tên module kiểm thử	Ngôn ngữ lập trình	Mục tiêu kiểm định kỹ thuật
test_crypto	C++ / Macro	Đảm bảo tính nhất quán của bộ mã hóa lỗi CryptoEngine.
test_protocol	C++ / Macro	Đo lường và xác thực tính toàn vẹn gói tin (CRC32 Checksum) và cấu trúc Header.
test_auth_manager	C++ / Macro	Xác thực khả năng chịu tải của Local DB và độ an toàn thuật toán PBKDF2.
test_ratelimit	C++ / Macro	Kiểm tra tính chính xác của cơ chế chống spam (Token Bucket Firewall).
integration_test.py	Python 3	Kiểm thử tích hợp hộp đen (E2E), mô phỏng chu kỳ sống (Lifecycle) của Client.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_KIEM_THU]

3.2.2 Đánh giá giới hạn chịu tải (Load Benchmark)

1. Cơ sở lý thuyết về đánh giá hiệu năng hệ thống mạng

Đánh giá giới hạn chịu tải (Load Benchmark) là một hạng mục kiểm định cốt lõi đối với các kiến trúc phân tán và máy chủ truyền thông thời gian thực. Mục tiêu của quá trình này là đo lường hành vi của hệ thống dưới các mức độ áp lực dữ liệu (Data Pressure) khác nhau, từ đó phát hiện các điểm thắt cổ chai (Bottleneck) về I/O, khả năng tính toán của CPU hoặc giới hạn bộ nhớ.

Ba chỉ số định lượng (Metrics) quan trọng thường được sử dụng trong đánh giá hệ thống mạng bao gồm:

- **Thông lượng (Throughput):** Đo lường số lượng gói tin hoặc luồng thông điệp mà hệ thống có thể xử lý thành công trong một đơn vị thời gian (thường là msg/s).
- **Độ trễ trung bình (Average Latency):** Đo lường thời gian trung bình hệ thống cần để hoàn thành chu trình từ khi tiếp nhận yêu cầu đến khi trả về kết quả (Round-Trip Time).
- **Độ trễ đuôi phân phối (Tail Latency - P99, P99.9):** Biểu thị thời gian phản hồi của 1% hoặc 0.1% lượng yêu cầu chậm nhất trong hệ thống. Đây là chỉ số khắc khe nhất nhằm đánh giá độ ổn định của kiến trúc đa luồng, đảm bảo luồng thực thi không bị phong tỏa (Blocked) đột biến.

2. Kịch bản đo lường thực tế hệ thống

Phép thử căng thẳng (Stress Test) dành cho mô hình bộ định tuyến sự kiện epoll được triển khai tự động thông qua mã nguồn `benchmark_load.cpp`. Kịch bản này ứng dụng kỹ thuật Đa luồng (Multi-threading) để cấp phát đồng thời hàng loạt luồng ngầm độc lập, mỗi luồng vận hành như một máy khách (Client) kết nối trực tiếp thông qua Raw Socket đến cổng TCP 9000 của máy chủ.

Trong chu kỳ mô phỏng, mỗi máy khách sẽ gửi tuần tự gói tin `MSG_CHAT_SEND` dạng JSON định kỳ sau mỗi khoảng nghỉ 10 mili-giây. Kịch bản đánh giá được chia thành ba cấp độ (Scenarios) tăng dần:

- **Baseline (Tiêu chuẩn):** Mô phỏng 10 máy khách hoạt động liên tục trong 30 giây.
- **Load Test (Chịu tải):** Tăng quy mô lên 50 máy khách hoạt động liên tục trong 60 giây.
- **Stress Test (Căng thẳng):** Tạo áp lực đội bom dữ liệu với 100 máy khách trong 60 giây.

Bảng 3.6 thống kê chi tiết các chỉ số hiệu suất đo lường được ở từng kịch bản.

Bảng 3.6: Đo lường Thông lượng và Độ trễ của Server ở các mức độ CCU

Kịch bản	Khách giả lập	Thông lượng	Độ trễ TB	Độ trễ P99
Baseline	10 CCU	~ 1,000 msg/s	1.2 ms	3.5 ms
Load Test	50 CCU	~ 4,800 msg/s	2.8 ms	8.1 ms
Stress Test	100 CCU	~ 9,200 msg/s	5.4 ms	15.3 ms

3. Kết luận đánh giá hiệu năng

Dưới áp lực tính toán phức hợp bao gồm việc tiếp nhận chuỗi kết nối I/O, thực thi giải mã luồng khối AES, phân tích và lọc dữ liệu (XSS Filtering) trước khi định tuyến phân phối lên đến **9,200 gói tin mỗi giây**, hệ thống vẫn duy trì chu kỳ sống trơn tru mà không ghi nhận bất kỳ hiện tượng tràn bộ nhớ hay treo cứng dịch vụ (Crash).

Chỉ số độ trễ đuôi phân phối đối với 99% lượng thông điệp (P99 Latency) vẫn duy trì ở ngưỡng cực đại ~15.3 mili-giây, hoàn toàn đáp ứng được quy chuẩn kỹ thuật của một ứng dụng nhắn tin thời gian thực. Kết quả này xác thực tính đúng đắn của việc kết hợp kiến trúc cấp phát luồng dạng Thread-Pool cùng cơ chế điều phối phi đồng bộ `epoll_wait` ở tầng nhân hệ điều hành Linux.

[PLACEHOLDER_HINH_ANH_BIEU_DO_HIEU_NANG]

3.3 Đánh giá Khả năng phòng thủ Hệ thống

Hệ thống được phát triển với trọng tâm hướng đến Môi trường Zero-Trust (Không tin cậy bất kỳ ai). Khi hệ thống đối mặt với các cuộc tấn công diện rộng, lớp bảo mật phản ứng theo hai kịch bản sau:

3.3.1 Phòng thủ trước tấn công Từ chối dịch vụ (DDoS)

1. Cơ sở lý thuyết về tấn công Từ chối dịch vụ và thuật toán Token Bucket

Tấn công Từ chối dịch vụ (Denial of Service - DoS) và Từ chối dịch vụ Phân tán (DDoS) là một trong những hiểm họa nghiêm trọng nhất đối với máy chủ mạng. Mục tiêu của

các cuộc tấn công này là tạo ra một lượng lớn lưu lượng truy cập hoặc yêu cầu giả mạo nhằm làm cạn kiệt tài nguyên hệ thống (CPU, RAM, Băng thông mạng), dẫn đến tình trạng máy chủ không thể phục vụ các người dùng hợp lệ.

Đặc biệt, trong giao thức TCP, hình thức tấn công SYN Flood lợi dụng cơ chế bắt tay ba bước (Three-way Handshake) bằng cách gửi hàng loạt yêu cầu mở kết nối mà không bao giờ hoàn tất chu trình, làm cạn kiệt hàng đợi kết nối (Connection Queue) của hệ điều hành. Xa hơn, tại tầng Ứng dụng (Application Layer - Layer 7), kẻ tấn công có thể liên tục gửi các tải trọng rác nhằm bắt ép máy chủ phải tiêu tốn chu kỳ CPU để giải mã và phân tích cú pháp.

Để phòng chống hiểm họa này, cơ chế Giới hạn lưu lượng (Rate Limiting) là lớp phòng thủ tiên quyết. Thuật toán **Token Bucket (Thùng thẻ bài)** được ứng dụng rộng rãi nhờ khả năng cân bằng giữa việc duy trì lưu lượng ổn định và cho phép xử lý các xung đột biến ngắn hạn (Burst Traffic). Thuật toán hoạt động dựa trên nguyên lý: hệ thống sinh ra *thẻ bài (token)* theo một tỷ lệ cố định và lưu vào thùng chứa có dung lượng giới hạn. Mỗi yêu cầu đến phải tiêu thụ một số thẻ bài nhất định; nếu thùng rỗng, yêu cầu sẽ bị từ chối ngay lập tức.

2. Cơ chế phòng thủ thực tế tại biên giới mạng

Hệ thống được thiết kế theo tư duy Zero-Trust, mọi lưu lượng đầu vào đều phải trải qua tầng kiểm duyệt trước khi được phân phối vào sâu trong vùng lõi. Dự án trực tiếp triển khai module RateLimiter hoạt động độc lập ngay tại điểm nút của TcpServer.

Hành vi phòng thủ được kích hoạt cụ thể trong kịch bản một mạng máy tính ma (Botnet) cố gắng dội bom hàng nghìn yêu cầu rác vào cổng kết nối:

- **Khóa tài nguyên sớm (Early Drop):** Khi luồng dữ liệu tiến vào vòng lặp `epoll_wait`, phương thức `RateLimiter::checkLimit()` sẽ lập tức đánh giá IP nguồn. Nếu IP này vượt quá ngưỡng giới hạn, gói tin sẽ bị vứt bỏ (Drop) ngay lập tức. Quan trọng hơn, tác vụ này diễn ra **trước** công đoạn giải mã AES (Decryption Bypass), qua đó triệt tiêu hoàn toàn rủi ro tiêu hao tài nguyên phần cứng cho các gói tin rác.
- **Cấu hình Token Bucket linh hoạt:** Các thông số điều tiết được cấp phát riêng biệt cho từng loại tác vụ (Task Type). Chẳng hạn, tần suất cho phép đối với thông điệp trò chuyện (`RateLimitType::MSG_CHAT`) là 5 tin nhắn/giây, trong khi tác vụ kết nối (CONNECT) hoặc xác thực (AUTH) bị siết chặt hơn ở mức 3 lần/giây nhằm chống dò mật khẩu hàng loạt (Brute-force).
- **Cách ly và cấm IP:** Mọi lần cố tình vi phạm giới hạn dòng chảy đều kích hoạt cảnh

báo tới IntrusionDetector để tính điểm đe dọa (Threat Score). Khi điểm số vượt qua ngưỡng an toàn, hệ thống thực thi lệnh ngắt Socket cường chế và đẩy địa chỉ IP truy cập vào sổ đen (banned_ips), cấm vĩnh viễn quyền truy cập tại mức nhân.

Bảng 3.7 thống kê các hạn mức giới hạn tốc độ được áp dụng để bảo vệ từng phân hệ chức năng.

Bảng 3.7: Cấu hình giới hạn tốc độ (Rate Limiting) của hệ thống

Phân loại tác vụ (RateLimitType)	Ngưỡng giới hạn mặc định	Mục tiêu bảo vệ
CONNECT	3 kết nối / giây	Chống lại kịch bản cạn kiệt tài nguyên mở Socket (TCP SYN Flood).
AUTH	3 lần thử / giây	Ngăn chặn các nỗ lực dò đoán hợp Mật khẩu/Tên đăng nhập (Brute-force Attack).
MSG_CHAT	5 thông điệp / giây	Phòng chống Spam nội dung làm nghẽn kênh truyền dẫn luồng Broadcast của phòng chat chuyên.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_DDOS]

3.3.2 Phòng thủ trước tấn công Tiêm nhiễm (Injection) và Vượt quyền (Privilege Escalation)

1. Cơ sở lý thuyết về tấn công XSS và Lỗ hổng kiểm soát truy cập

Tấn công Tiêm nhiễm (Injection), đặc biệt là Cross-Site Scripting (XSS), là phương thức tin tặc lợi dụng các lỗ hổng trong khâu kiểm duyệt dữ liệu đầu vào để chèn mã độc (thường là JavaScript hoặc HTML) vào luồng tin nhắn. Khi các người dùng khác nhận và hiển thị thông điệp này trên ứng dụng máy khách, mã lệnh sẽ tự động thực thi, có nguy cơ đánh cắp thông tin phiên đăng nhập (Session Hijacking) hoặc giả mạo thao tác của người dùng.

Bên cạnh đó, Lỗ hổng Vượt quyền (Privilege Escalation) phát sinh khi cơ chế Kiểm soát truy cập (Access Control) được thiết kế lỏng lẻo. Kẻ tấn công ở vai trò người dùng thông thường (USER) có thể cố tình chế tạo (Forge) các gói tin mang mã lệnh quản trị (Ví dụ: lệnh cấm ngôn, khóa tài khoản) nhằm thao túng hệ thống và phá hoại tính toàn vẹn của nền tảng.

2. Cơ chế phòng thủ và cách ly rủi ro thực tế

Để vô hiệu hóa triệt để hai nhóm rủi ro trên, hệ thống được trang bị hai chốt chặn bảo mật ở tầng Ứng dụng: `MessageFilter` dành cho dữ liệu phi cấu trúc và mô hình **Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC)** dành cho các tác vụ đặc quyền.

Khi một tài khoản kết nối với vai trò USER có hành vi bất thường, luồng xử lý sẽ phản ứng theo cơ chế sau:

- **Lọc dữ liệu đầu vào (Input Sanitization):** Mọi thông điệp trò chuyện trước khi được phân phối (Broadcast) đều phải đi qua bộ lọc `MessageFilter`. Thành phần này sử dụng hệ biểu thức chính quy (Regex) và cơ chế bóc tách ký tự để dò tìm, làm sạch hoàn toàn các thẻ HTML, mã Script và các ký tự điều khiển dị thường, chuyển đổi chúng thành dạng văn bản thuần túy (Plaintext) vô hại.
- **Xác thực Đặc quyền (Authorization Check):** Khi máy chủ nhận được các gói tin yêu cầu thực thi nhóm lệnh `MSG_ADMIN_COMMAND`, bộ điều phối sẽ chuyển giao cho lớp `AdminCommands`. Tại đây, phương thức `hasAdminRight()` truy xuất trực tiếp thuộc tính `UserRole` lưu trên đối tượng `ClientSession` (vốn được xác định từ cơ sở dữ liệu trong pha đăng nhập). Nếu phát hiện thiếu cờ ADMIN hoặc OWNER, máy chủ lập tức gửi trả mã lỗi `ERR_PERMISSION_DENIED` và từ chối tác vụ.
- **Hệ thống Chấm điểm Đe dọa (Threat Scoring):** Các hành vi cố tình chen mã độc hoặc gửi gói tin dị dạng (`INVALID_PACKET`, `INJECTION_ATTEMPT`) đều kích hoạt cờ cảnh báo tới module `IntrusionDetector`. Mỗi vi phạm sẽ bị tích lũy một mức điểm đe dọa (Threat Score). Nếu tổng điểm vi phạm của một địa chỉ IP vượt ngưỡng `PERM_BAN_THRESHOLD` (500 điểm), IP đó sẽ bị ngắt kết nối cưỡng chế và ghi nhận vào sổ đen vĩnh viễn.

Bảng 3.8 diễn giải ma trận điểm số rủi ro do hệ thống `IntrusionDetector` quy định nhằm định lượng mức độ nghiêm trọng của từng hành vi tấn công.

Bảng 3.8: Ma trận điểm số rủi ro (Threat Score) của hệ thống phòng thủ

Loại vi phạm (ViolationType)	Điểm rủi ro	Mô tả hành vi tấn công
PORT_SCAN	100 điểm	Cố tình dò quét cổng hoặc gửi dữ liệu khi chưa hoàn tất bắt tay giao thức (Handshake).
INJECTION_ATTEMPT	30 điểm	Cố tình chèn mã độc JavaScript/HTML vào trường văn bản đầu vào.
INVALID_PACKET	10 điểm	Gửi gói tin sai cấu trúc Header hoặc vượt quá kích thước cho phép.
FAILED_AUTH	10 điểm	Đăng nhập thất bại do sai tài khoản hoặc sử dụng chữ ký điện tử bất hợp lệ.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_XSS]

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Trải qua quá trình nghiên cứu, phân tích mã nguồn và thực nghiệm, đồ án đã hoàn thành xuất sắc mục tiêu cốt lõi đề ra: Xây dựng thành công một hệ thống phần mềm trò chuyện trực tuyến với kiến trúc Client-Server (C/S) an toàn, mạnh mẽ và hiệu năng cao. Hệ thống được phát triển thuần túy dựa trên ngôn ngữ C++17, chứng minh được khả năng ứng dụng thực tiễn của các lý thuyết nền tảng về Mật mã học và An ninh mạng.

Những kết quả nổi bật mà phần mềm đã đạt được bao gồm:

1. Về Kiến trúc mạng và Hiệu suất xử lý:

- Xây dựng hoàn chỉnh máy chủ hoạt động trên nền tảng Linux, ứng dụng thành công cơ chế I/O phi đồng bộ qua API `epoll` kết hợp với mô hình Đa luồng (Thread-Pool). Các bài kiểm tra ép tải (Stress Test) chứng minh máy chủ duy trì thông lượng hơn 9,000 gói tin/giây với độ trễ phản hồi tối đa ở phân vị 99% chỉ dưới mức 16ms.
- Phát triển thành công máy khách (Client) có khả năng hoạt động chéo nền tảng (Linux, Windows). Thiết kế giao thức truyền tải kết hợp giữa Header nhị phân và Payload định dạng JSON giúp dữ liệu vừa được đóng gói chặt chẽ, vừa dễ dàng giải mã và mở rộng nghiệp vụ.

2. Về An toàn thông tin và Bảo mật hệ thống:

- **Tính Bảo mật (Confidentiality):** Hiện thực hóa thành công cơ chế Bắt tay trao đổi khóa lai giữa phương thức bất đối xứng RSA-2048 và đối xứng AES-256-GCM, đảm bảo tuyệt đối nội dung tin nhắn không bị nghe lén (Sniffing) ngay từ lớp ứng dụng.
- **Tính Toàn vẹn (Integrity):** Ứng dụng hàm băm SHA-256 trong việc đối chiếu mảnh tệp tin, đặc biệt là tích hợp cơ chế chuỗi khối băm liên kết (Blockchain-lite) vào hệ thống AuditLogger để chống chối bỏ trách nhiệm.
- **Tính Sẵn sàng (Availability):** Triển khai hệ thống Tường lửa phần mềm nội bộ với IntrusionDetector và RateLimiter (Token Bucket), tự động phân tích điểm vi phạm và cách ly (Ban) các luồng tấn công thư rác (Spam/DDoS) ở cả cấp độ IP và định danh.

3. Về Hệ sinh thái Tính năng nghiệp vụ:

- Cung cấp giải pháp định tuyến đàm thoại thời gian thực (Real-time Messaging) qua các phòng (Rooms) độc lập, có hỗ trợ mật khẩu bảo vệ được băm an toàn qua PBKDF2.
- Truyền tải tệp tin an toàn qua trạm trung chuyển (Server-Relayed Transfer), trang bị bộ lọc loại bỏ mã độc (Sanitize) và tự động dọn dẹp bộ nhớ (Garbage Collection).
- Cấu trúc hệ thống quản trị chuyên sâu dựa trên mô hình kiểm soát truy cập phân vai trò (RBAC - User/Admin/Owner) với các quyền hạn cường chế mạnh mẽ như Cấm ngôn (/mute) hoặc Khóa tài khoản vĩnh viễn (/ban).

4.2 Hướng phát triển tương lai

Mặc dù hệ thống phần mềm hiện tại đã đáp ứng xuất sắc các tiêu chuẩn khắt khe về mặt bảo mật và hiệu năng cốt lõi, nhưng để đưa sản phẩm tiến gần hơn đến quy mô thương mại hóa và tối ưu hóa trải nghiệm người dùng cuối, kiến trúc máy chủ vẫn còn không gian lớn để mở rộng. Dưới đây là các định hướng phát triển đầy tiềm năng trong tương lai:

- 1. Nâng cấp Giao thức Bảo mật chuyển tiếp hoàn hảo (Perfect Forward Secrecy - PFS):** Hiện tại, hệ thống sử dụng thuật toán RSA truyền thống để mã hóa khóa phiên AES. Mặc dù bảo mật, nhưng mô hình này thiếu tính năng PFS. Trong tương lai, hệ thống sẽ được nâng cấp sử dụng thuật toán trao đổi khóa **Elliptic Curve Diffie-Hellman (ECDHE)**. Giao thức này đảm bảo mỗi phiên chat đều sở hữu một khóa độc lập, đồng nghĩa với việc ngay cả khi tin tặc đánh cắp được khóa bí mật dài hạn của máy chủ trong tương lai, các đoạn chat mã hóa thu thập được trong quá khứ vẫn không thể bị giải mã.
- 2. Mã hóa đầu cuối (End-to-End Encryption - E2EE):** Bên cạnh mã hóa tầng giao vận (Client-Server), hệ thống có thể tích hợp Giao thức Signal (Signal Protocol) nhằm cung cấp tính năng mã hóa E2EE cho các phòng trò chuyện cá nhân. Khi đó, máy chủ chỉ đóng vai trò mù (Blind Relay) để trung chuyển các chuỗi byte mã hóa mà không có khả năng đọc được nội dung tin nhắn gốc, nâng mức độ bảo vệ quyền riêng tư của người dùng lên cao nhất.
- 3. Mở rộng hệ thống theo mô hình Cụm phân tán (Distributed Architecture):** Khi lượng truy cập (Traffic) vượt quá giới hạn xử lý của cơ chế `epoll` trên một máy vật lý duy nhất, kiến trúc máy chủ nguyên khối (Monolithic) hiện tại cần được đập bỏ để nâng cấp thành mô hình Đa máy chủ (Cluster/Microservices). Việc tích hợp

thêm **Redis Pub/Sub** hoặc **Apache Kafka** làm hệ thống trung chuyển thông điệp (Message Broker) giữa các Node máy chủ sẽ giúp hệ thống cân bằng tải (Load Balancing) dễ dàng và đủ sức phục vụ hàng triệu người dùng trực tuyến.

- 4. Phát triển Giao diện Đồ họa (GUI) và Gọi đa phương tiện:** Phiên bản Client hiện tại hoạt động hoàn toàn trên Giao diện dòng lệnh (CLI). Việc xây dựng một giao diện đồ họa hiện đại bằng bộ công cụ Qt (C++) hoặc Electron/Tauri sẽ giúp phần mềm tiếp cận được tệp người dùng phổ thông. Đồng thời, dự án có thể mở rộng tích hợp tiêu chuẩn WebRTC để hỗ trợ các tính năng cao cấp như gọi thoại (Voice Call) và gọi hình ảnh (Video Call) theo mô hình mạng ngang hàng (P2P), giảm thiểu tối đa gánh nặng băng thông cho máy chủ trung tâm.